CHAPTER 13

COPY CONTROL

CONTENTS

As we saw in Chapter 7, each class defines a new type and defines the operations that objects of that type can perform. In that chapter, we also learned that classes can define constructors, which control what happens when objects of the class type are created.

In this chapter we'll learn how classes can control what happens when objects of the class type are copied, assigned, moved, or destroyed. Classes control these actions through special member functions: the copy constructor, move constructor, copy-assignment operator, move-assignment operator, and destructor.

When we define a class, we specify—explicitly or implicitly—what happens when objects of that class type are copied, moved, assigned, and destroyed. A class controls these operations by defining five special member functions: **copy constructor**, **copy-assignment operator**, **move constructor**, **move-assignment operator**, and **destructor**. The copy and move constructors define what happens when an object is initialized from another object of the same type. The copy- and move-assignment operators define what happens when we assign an object of a class type to another object of that same class type. The destructor defines what happens when an object of the type ceases to exist. Collectively, we'll refer to these operations as **copy control**.

If a class does not define all of the copy-control members, the compiler automatically defines the missing operations. As a result, many classes can ignore copy control (§ 7.1.5, p. 267). However, for some classes, relying on the default definitions leads to disaster. Frequently, the hardest part of implementing copy-control operations is recognizing when we need to define them in the first place.

Copy control is an essential part of defining any C++ class. Programmers new to C++ are often confused by having to define what happens when objects are copied, moved, assigned, or destroyed. This confusion is compounded because if we do not explicitly define these operations, the compiler defines them for us—although the compiler-defined versions might not behave as we intend.

13.1 Copy, Assign, and Destroy

We'll start by covering the most basic operations, which are the copy constructor, copy-assignment operator, and destructor. We'll cover the move operations (which were introduced by the new standard) in § 13.6 (p. 531).

13.1.1 The Copy Constructor

A constructor is the copy constructor if its first parameter is a reference to the class type and any additional parameters have default values:

```
class Foo {
public:
   Foo();               // default constructor
   Foo(const Foo&);     // copy constructor
   // ...
};
```

For reasons we'll explain shortly, the first parameter must be a reference type. That parameter is almost always a reference to `const`, although we can define the copy constructor to take a reference to non`const`. The copy constructor is used implicitly in several circumstances. Hence, the copy constructor usually should not be `explicit` (§ 7.5.4, p. 296).

The Synthesized Copy Constructor

When we do not define a copy constructor for a class, the compiler synthesizes one for us. Unlike the synthesized default constructor (§ 7.1.4, p. 262), a copy constructor is synthesized even if we define other constructors.

As we'll see in § 13.1.6 (p. 508), the **synthesized copy constructor** for some classes prevents us from copying objects of that class type. Otherwise, the synthesized copy constructor **memberwise copies** the members of its argument into the object being created (§ 7.1.5, p. 267). The compiler copies each nonstatic member in turn from the given object into the one being created.

The type of each member determines how that member is copied: Members of class type are copied by the copy constructor for that class; members of built-in type are copied directly. Although we cannot directly copy an array (§ 3.5.1, p. 114), the synthesized copy constructor copies members of array type by copying each element. Elements of class type are copied by using the elements' copy constructor.

As an example, the synthesized copy constructor for our `Sales_data` class is equivalent to:

```
class Sales_data {
public:
    // other members and constructors as before
    // declaration equivalent to the synthesized copy constructor
    Sales_data(const Sales_data&);
private:
    std::string bookNo;
    int units_sold = 0;
    double revenue = 0.0;
};
// equivalent to the copy constructor that would be synthesized for Sales_data
Sales_data::Sales_data(const Sales_data &orig):
    bookNo(orig.bookNo),          // uses the string copy constructor
    units_sold(orig.units_sold), // copies orig.units_sold
    revenue(orig.revenue)         // copies orig.revenue
    {    }                        // empty body
```

Copy Initialization

We are now in a position to fully understand the differences between direct initialization and copy initialization (§ 3.2.1, p. 84):

```
string dots(10, '.');               // direct initialization
string s(dots);                     // direct initialization
string s2 = dots;                   // copy initialization
string null_book = "9-999-99999-9"; // copy initialization
string nines = string(100, '9');    // copy initialization
```

When we use direct initialization, we are asking the compiler to use ordinary function matching (§ 6.4, p. 233) to select the constructor that best matches the arguments we provide. When we use **copy initialization**, we are asking the compiler to copy the right-hand operand into the object being created, converting that operand if necessary (§ 7.5.4, p. 294).

Copy initialization ordinarily uses the copy constructor. However, as we'll see in § 13.6.2 (p. 534), if a class has a move constructor, then copy initialization sometimes uses the move constructor instead of the copy constructor. For now, what's useful to know is when copy initialization happens and that copy initialization requires either the copy constructor or the move constructor.

Copy initialization happens not only when we define variables using an =, but also when we

- Pass an object as an argument to a parameter of nonreference type
- Return an object from a function that has a nonreference return type
- Brace initialize the elements in an array or the members of an aggregate class (§ 7.5.5, p. 298)

Some class types also use copy initialization for the objects they allocate. For example, the library containers copy initialize their elements when we initialize the container, or when we call an `insert` or `push` member (§ 9.3.1, p. 342). By contrast, elements created by an `emplace` member are direct initialized (§ 9.3.1, p. 345).

Parameters and Return Values

During a function call, parameters that have a nonreference type are copy initialized (§ 6.2.1, p. 209). Similarly, when a function has a nonreference return type, the return value is used to copy initialize the result of the call operator at the call site (§ 6.3.2, p. 224).

The fact that the copy constructor is used to initialize nonreference parameters of class type explains why the copy constructor's own parameter must be a reference. If that parameter were not a reference, then the call would never succeed—to call the copy constructor, we'd need to use the copy constructor to copy the argument, but to copy the argument, we'd need to call the copy constructor, and so on indefinitely.

Constraints on Copy Initialization

As we've seen, whether we use copy or direct initialization matters if we use an initializer that requires conversion by an `explicit` constructor (§ 7.5.4, p. 296):

```
vector<int> v1(10);   // ok: direct initialization
vector<int> v2 = 10; // error: constructor that takes a size is explicit
void f(vector<int>); // f's parameter is copy initialized
f(10); // error: can't use an explicit constructor to copy an argument
f(vector<int>(10));   // ok: directly construct a temporary vector from an int
```

Directly initializing `v1` is fine, but the seemingly equivalent copy initialization of `v2` is an error, because the `vector` constructor that takes a single size parameter is `explicit`. For the same reasons that we cannot copy initialize `v2`, we cannot implicitly use an `explicit` constructor when we pass an argument or return a value from a function. If we want to use an `explicit` constructor, we must do so explicitly, as in the last line of the example above.

The Compiler Can Bypass the Copy Constructor

During copy initialization, the compiler is permitted (but not obligated) to skip the copy/move constructor and create the object directly. That is, the compiler is permitted to rewrite

```
string null_book = "9-999-99999-9"; //  copy initialization
```

into

```
string null_book("9-999-99999-9"); //  compiler omits the copy constructor
```

However, even if the compiler omits the call to the copy/move constructor, the copy/move constructor must exist and must be accessible (e.g., not `private`) at that point in the program.

EXERCISES SECTION 13.1.1

Exercise 13.1: What is a copy constructor? When is it used?

Exercise 13.2: Explain why the following declaration is illegal:

```
Sales_data::Sales_data(Sales_data rhs);
```

Exercise 13.3: What happens when we copy a `StrBlob`? What about `StrBlobPtrs`?

Exercise 13.4: Assuming `Point` is a class type with a `public` copy constructor, identify each use of the copy constructor in this program fragment:

```
Point global;
Point foo_bar(Point arg)
{
    Point local = arg, *heap = new Point(global);
    *heap = local;
    Point pa[ 4 ] = { local, *heap };
    return *heap;
}
```

Exercise 13.5: Given the following sketch of a class, write a copy constructor that copies all the members. Your constructor should dynamically allocate a new `string` (§ 12.1.2, p. 458) and copy the object to which `ps` points, rather than copying `ps` itself.

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
private:
    std::string *ps;
    int    i;
};
```

13.1.2 The Copy-Assignment Operator

Just as a class controls how objects of that class are initialized, it also controls how objects of its class are assigned:

```
Sales_data trans, accum;
trans = accum; //  uses the Sales_data copy-assignment operator
```

As with the copy constructor, the compiler synthesizes a copy-assignment operator if the class does not define its own.

Introducing Overloaded Assignment

Before we look at the synthesized assignment operator, we need to know a bit about **overloaded operators**, which we cover in detail in Chapter 14.

Overloaded operators are functions that have the name `operator` followed by the symbol for the operator being defined. Hence, the assignment operator is a function named `operator=`. Like any other function, an operator function has a return type and a parameter list.

The parameters in an overloaded operator represent the operands of the operator. Some operators, assignment among them, must be defined as member functions. When an operator is a member function, the left-hand operand is bound to the implicit `this` parameter (§ 7.1.2, p. 257). The right-hand operand in a binary operator, such as assignment, is passed as an explicit parameter.

The copy-assignment operator takes an argument of the same type as the class:

```
class Foo {
public:
    Foo& operator=(const Foo&); //  assignment operator
   // ...
};
```

To be consistent with assignment for the built-in types (§ 4.4, p. 145), assignment operators usually return a reference to their left-hand operand. It is also worth noting that the library generally requires that types stored in a container have assignment operators that return a reference to the left-hand operand.

Assignment operators ordinarily should return a reference to their left-hand operand.

The Synthesized Copy-Assignment Operator

Just as it does for the copy constructor, the compiler generates a **synthesized copy-assignment operator** for a class if the class does not define its own. Analogously to the copy constructor, for some classes the synthesized copy-assignment operator disallows assignment (§ 13.1.6, p. 508). Otherwise, it assigns each nonstatic member of the right-hand object to the corresponding member of the left-hand object using the copy-assignment operator for the type of that member. Array members are assigned by assigning each element of the array. The synthesized copy-assignment operator returns a reference to its left-hand object.

As an example, the following is equivalent to the synthesized `Sales_data` copy-assignment operator:

```
// equivalent to the synthesized copy-assignment operator
Sales_data&
Sales_data::operator=(const Sales_data &rhs)
{
    bookNo = rhs.bookNo;           // calls the string::operator=
    units_sold = rhs.units_sold;   // uses the built-in int assignment
    revenue = rhs.revenue;         // uses the built-in double assignment
    return *this;                  // return a reference to this object
}
```

EXERCISES SECTION 13.1.2

Exercise 13.6: What is a copy-assignment operator? When is this operator used? What does the synthesized copy-assignment operator do? When is it synthesized?

Exercise 13.7: What happens when we assign one `StrBlob` to another? What about `StrBlobPtrs`?

Exercise 13.8: Write the assignment operator for the `HasPtr` class from exercise 13.5 in § 13.1.1 (p. 499). As with the copy constructor, your assignment operator should copy the object to which `ps` points.

13.1.3 The Destructor

The destructor operates inversely to the constructors: Constructors initialize the nonstatic data members of an object and may do other work; destructors do whatever work is needed to free the resources used by an object and destroy the nonstatic data members of the object.

The destructor is a member function with the name of the class prefixed by a tilde (~). It has no return value and takes no parameters:

```
class Foo {
public:
    ~Foo();    // destructor
   // ...
};
```

Because it takes no parameters, it cannot be overloaded. There is always only one destructor for a given class.

What a Destructor Does

Just as a constructor has an initialization part and a function body (§ 7.5.1, p. 288), a destructor has a function body and a destruction part. In a constructor, members are initialized before the function body is executed, and members are initialized

in the same order as they appear in the class. In a destructor, the function body is executed first and then the members are destroyed. Members are destroyed in reverse order from the order in which they were initialized.

The function body of a destructor does whatever operations the class designer wishes to have executed subsequent to the last use of an object. Typically, the destructor frees resources an object allocated during its lifetime.

In a destructor, there is nothing akin to the constructor initializer list to control how members are destroyed; the destruction part is implicit. What happens when a member is destroyed depends on the type of the member. Members of class type are destroyed by running the member's own destructor. The built-in types do not have destructors, so nothing is done to destroy members of built-in type.

The implicit destruction of a member of built-in pointer type does *not* `delete` the object to which that pointer points.

Unlike ordinary pointers, the smart pointers (§ 12.1.1, p. 452) are class types and have destructors. As a result, unlike ordinary pointers, members that are smart pointers are automatically destroyed during the destruction phase.

When a Destructor Is Called

The destructor is used automatically whenever an object of its type is destroyed:

- Variables are destroyed when they go out of scope.
- Members of an object are destroyed when the object of which they are a part is destroyed.
- Elements in a container—whether a library container or an array—are destroyed when the container is destroyed.
- Dynamically allocated objects are destroyed when the `delete` operator is applied to a pointer to the object (§ 12.1.2, p. 460).
- Temporary objects are destroyed at the end of the full expression in which the temporary was created.

Because destructors are run automatically, our programs can allocate resources and (usually) not worry about when those resources are released.

For example, the following fragment defines four `Sales_data` objects:

```
{ // new scope
    // p and p2 point to dynamically allocated objects
    Sales_data *p = new Sales_data;       // p is a built-in pointer
    auto p2 = make_shared<Sales_data>(); // p2 is a shared_ptr
    Sales_data item(*p);      // copy constructor copies *p into item
    vector<Sales_data> vec;   // local object
    vec.push_back(*p2);       // copies the object to which p2 points
    delete p;                 // destructor called on the object pointed to by p
} // exit local scope; destructor called on item, p2, and vec
  // destroying p2 decrements its use count; if the count goes to 0, the object is freed
  // destroying vec destroys the elements in vec
```

Each of these objects contains a `string` member, which allocates dynamic memory to contain the characters in its `bookNo` member. However, the only memory our code has to manage directly is the object we directly allocated. Our code directly frees only the dynamically allocated object bound to `p`.

The other `Sales_data` objects are automatically destroyed when they go out of scope. When the block ends, `vec`, `p2`, and `item` all go out of scope, which means that the `vector`, `shared_ptr`, and `Sales_data` destructors will be run on those objects, respectively. The `vector` destructor will destroy the element we pushed onto `vec`. The `shared_ptr` destructor will decrement the reference count of the object to which `p2` points. In this example, that count will go to zero, so the `shared_ptr` destructor will `delete` the `Sales_data` object that `p2` allocated.

In all cases, the `Sales_data` destructor implicitly destroys the `bookNo` member. Destroying `bookNo` runs the `string` destructor, which frees the memory used to store the ISBN.

The destructor is *not* run when a reference or a pointer to an object goes out of scope.

The Synthesized Destructor

The compiler defines a **synthesized destructor** for any class that does not define its own destructor. As with the copy constructor and the copy-assignment operator, for some classes, the synthesized destructor is defined to disallow objects of the type from being destroyed (§ 13.1.6, p. 508). Otherwise, the synthesized destructor has an empty function body.

For example, the synthesized `Sales_data` destructor is equivalent to:

```
class Sales_data {
public:
    // no work to do other than destroying the members, which happens automatically
     ~Sales_data() { }
    // other members as before
};
```

The members are automatically destroyed after the (empty) destructor body is run. In particular, the `string` destructor will be run to free the memory used by the `bookNo` member.

It is important to realize that the destructor body does not directly destroy the members themselves. Members are destroyed as part of the implicit destruction phase that follows the destructor body. A destructor body executes *in addition to* the memberwise destruction that takes place as part of destroying an object.

13.1.4 The Rule of Three/Five

As we've seen, there are three basic operations to control copies of class objects: the copy constructor, copy-assignment operator, and destructor. Moreover, as we'll see in § 13.6 (p. 531), under the new standard, a class can also define a move constructor and move-assignment operator.

Exercises Section 13.1.3

Exercise 13.9: What is a destructor? What does the synthesized destructor do? When is a destructor synthesized?

Exercise 13.10: What happens when a `StrBlob` object is destroyed? What about a `StrBlobPtr`?

Exercise 13.11: Add a destructor to your `HasPtr` class from the previous exercises.

Exercise 13.12: How many destructor calls occur in the following code fragment?

```
bool fcn(const Sales_data *trans, Sales_data accum)
{
    Sales_data item1(*trans), item2(accum);
    return item1.isbn() != item2.isbn();
}
```

Exercise 13.13: A good way to understand copy-control members and constructors is to define a simple class with these members in which each member prints its name:

```
struct X {
    X() {std::cout << "X()" << std::endl;}
    X(const X&) {std::cout << "X(const X&)" << std::endl;}
};
```

Add the copy-assignment operator and destructor to `X` and write a program using `X` objects in various ways: Pass them as nonreference and reference parameters; dynamically allocate them; put them in containers; and so forth. Study the output until you are certain you understand when and why each copy-control member is used. As you read the output, remember that the compiler can omit calls to the copy constructor.

There is no requirement that we define all of these operations: We can define one or two of them without having to define all of them. However, ordinarily these operations should be thought of as a unit. In general, it is unusual to need one without needing to define them all.

Classes That Need Destructors Need Copy and Assignment

One rule of thumb to use when you decide whether a class needs to define its own versions of the copy-control members is to decide first whether the class needs a destructor. Often, the need for a destructor is more obvious than the need for the copy constructor or assignment operator. If the class needs a destructor, it almost surely needs a copy constructor and copy-assignment operator as well.

The `HasPtr` class that we have used in the exercises is a good example (§ 13.1.1, p. 499). That class allocates dynamic memory in its constructor. The synthesized destructor will not `delete` a data member that is a pointer. Therefore, this class needs to define a destructor to free the memory allocated by its constructor.

What may be less clear—but what our rule of thumb tells us—is that `HasPtr` also needs a copy constructor and copy-assignment operator.

Consider what would happen if we gave `HasPtr` a destructor but used the synthesized versions of the copy constructor and copy-assignment operator:

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    ~HasPtr() { delete ps; }
    // WRONG: HasPtr needs a copy constructor and copy-assignment operator
    // other members as before
};
```

In this version of the class, the memory allocated in the constructor will be freed when a `HasPtr` object is destroyed. Unfortunately, we have introduced a serious bug! This version of the class uses the synthesized versions of copy and assignment. Those functions copy the pointer member, meaning that multiple `HasPtr` objects may be pointing to the same memory:

```
HasPtr f(HasPtr hp)   // HasPtr passed by value, so it is copied
{
    HasPtr ret = hp; // copies the given HasPtr
    // process ret
    return ret;      // ret and hp are destroyed
}
```

When `f` returns, both `hp` and `ret` are destroyed and the `HasPtr` destructor is run on each of these objects. That destructor will `delete` the pointer member in `ret` and in `hp`. But these objects contain the same pointer value. This code will `delete` that pointer twice, which is an error (§ 12.1.2, p. 462). What happens is undefined.

In addition, the caller of `f` may still be using the object that was passed to `f`:

```
HasPtr p("some values");
f(p);         // when f completes, the memory to which p.ps points is freed
HasPtr q(p); // now both p and q point to invalid memory!
```

The memory to which `p` (and `q`) points is no longer valid. It was returned to the system when `hp` (or `ret`!) was destroyed.

If a class needs a destructor, it almost surely also needs the copy-assignment operator and a copy constructor.

Classes That Need Copy Need Assignment, and Vice Versa

Although many classes need to define all of (or none of) the copy-control members, some classes have work that needs to be done to copy or assign objects but has no need for the destructor.

As an example, consider a class that gives each object its own, unique serial number. Such a class would need a copy constructor to generate a new, distinct serial number for the object being created. That constructor would copy all the other data members from the given object. This class would also need its own

copy-assignment operator to avoid assigning to the serial number of the left-hand object. However, this class would have no need for a destructor.

This example gives rise to a second rule of thumb: If a class needs a copy constructor, it almost surely needs a copy-assignment operator. And vice versa—if the class needs an assignment operator, it almost surely needs a copy constructor as well. Nevertheless, needing either the copy constructor or the copy-assignment operator does not (necessarily) indicate the need for a destructor.

Exercises Section 13.1.4

Exercise 13.14: Assume that `numbered` is a class with a default constructor that generates a unique serial number for each object, which is stored in a data member named `mysn`. Assuming `numbered` uses the synthesized copy-control members and given the following function:

```
void f (numbered s) { cout << s.mysn << endl; }
```

what output does the following code produce?

```
numbered a, b = a, c = b;
f(a); f(b); f(c);
```

Exercise 13.15: Assume `numbered` has a copy constructor that generates a new serial number. Does that change the output of the calls in the previous exercise? If so, why? What output gets generated?

Exercise 13.16: What if the parameter in `f` were `const numbered&`? Does that change the output? If so, why? What output gets generated?

Exercise 13.17: Write versions of `numbered` and `f` corresponding to the previous three exercises and check whether you correctly predicted the output.

13.1.5 Using = `default`

C++ 11

We can explicitly ask the compiler to generate the synthesized versions of the copy-control members by defining them as `= default` (§ 7.1.4, p. 264):

```
class Sales_data {
public:
    // copy control; use defaults
    Sales_data() = default;
    Sales_data(const Sales_data&) = default;
    Sales_data& operator=(const Sales_data &);
    ~Sales_data() = default;
    // other members as before
};
Sales_data& Sales_data::operator=(const Sales_data&) = default;
```

When we specify `= default` on the declaration of the member inside the class body, the synthesized function is implicitly inline (just as is any other member

function defined in the body of the class). If we do not want the synthesized member to be an inline function, we can specify = default on the member's definition, as we do in the definition of the copy-assignment operator.

We can use = default only on member functions that have a synthesized version (i.e., the default constructor or a copy-control member).

13.1.6 Preventing Copies

Most classes should define—either implicitly or explicitly—the default and copy constructors and the copy-assignment operator.

Although most classes should (and generally do) define a copy constructor and a copy-assignment operator, for some classes, there really is no sensible meaning for these operations. In such cases, the class must be defined so as to prevent copies or assignments from being made. For example, the iostream classes prevent copying to avoid letting multiple objects write to or read from the same IO buffer. It might seem that we could prevent copies by not defining the copy-control members. However, this strategy doesn't work: If our class doesn't define these operations, the compiler will synthesize them.

Defining a Function as Deleted

Under the new standard, we can prevent copies by defining the copy constructor and copy-assignment operator as **deleted functions**. A deleted function is one that is declared but may not be used in any other way. We indicate that we want to define a function as deleted by following its parameter list with = delete:

C++ 11

```
struct NoCopy {
    NoCopy() = default;     // use the synthesized default constructor
    NoCopy(const NoCopy&) = delete;                // no copy
    NoCopy &operator=(const NoCopy&) = delete;   // no assignment
    ~NoCopy() = default;    // use the synthesized destructor
    // other members
};
```

The = delete signals to the compiler (and to readers of our code) that we are intentionally *not defining* these members.

Unlike = default, = delete must appear on the first declaration of a deleted function. This difference follows logically from the meaning of these declarations. A defaulted member affects only what code the compiler generates; hence the = default is not needed until the compiler generates code. On the other hand, the compiler needs to know that a function is deleted in order to prohibit operations that attempt to use it.

Also unlike = default, we can specify = delete on any function (we can use = default only on the default constructor or a copy-control member that the compiler can synthesize). Although the primary use of deleted functions is to

suppress the copy-control members, deleted functions are sometimes also useful when we want to guide the function-matching process.

The Destructor Should Not be a Deleted Member

It is worth noting that we did not delete the destructor. If the destructor is deleted, then there is no way to destroy objects of that type. The compiler will not let us define variables or create temporaries of a type that has a deleted destructor. Moreover, we cannot define variables or temporaries of a class that has a member whose type has a deleted destructor. If a member has a deleted destructor, then that member cannot be destroyed. If a member can't be destroyed, the object as a whole can't be destroyed.

Although we cannot define variables or members of such types, we can dynamically allocate objects with a deleted destructor. However, we cannot free them:

```
struct NoDtor {
    NoDtor() = default;   // use the synthesized default constructor
    ~NoDtor() = delete;   // we can't destroy objects of type NoDtor
};
NoDtor nd;   // error: NoDtor destructor is deleted
NoDtor *p = new NoDtor();   // ok: but we can't delete p
delete p; // error: NoDtor destructor is deleted
```

It is not possible to define an object or delete a pointer to a dynamically allocated object of a type with a deleted destructor.

The Copy-Control Members May Be Synthesized as Deleted

As we've seen, if we do not define the copy-control members, the compiler defines them for us. Similarly, if a class defines no constructors, the compiler synthesizes a default constructor for that class (§ 7.1.4, p. 262). For some classes, the compiler defines these synthesized members as deleted functions:

- The synthesized destructor is defined as deleted if the class has a member whose own destructor is deleted or is inaccessible (e.g., `private`).
- The synthesized copy constructor is defined as deleted if the class has a member whose own copy constructor is deleted or inaccessible. It is also deleted if the class has a member with a deleted or inaccessible destructor.
- The synthesized copy-assignment operator is defined as deleted if a member has a deleted or inaccessible copy-assignment operator, or if the class has a `const` or reference member.
- The synthesized default constructor is defined as deleted if the class has a member with a deleted or inaccessible destructor; or has a reference member that does not have an in-class initializer (§ 2.6.1, p. 73); or has a `const` member whose type does not explicitly define a default constructor and that member does not have an in-class initializer.

In essence, these rules mean that if a class has a data member that cannot be default constructed, copied, assigned, or destroyed, then the corresponding member will be a deleted function.

It may be surprising that a member that has a deleted or inaccessible destructor causes the synthesized default and copy constructors to be defined as deleted. The reason for this rule is that without it, we could create objects that we could not destroy.

It should not be surprising that the compiler will not synthesize a default constructor for a class with a reference member or a `const` member that cannot be default constructed. Nor should it be surprising that a class with a `const` member cannot use the synthesized copy-assignment operator: After all, that operator attempts to assign to every member. It is not possible to assign a new value to a `const` object.

Although we can assign a new value to a reference, doing so changes the value of the object to which the reference refers. If the copy-assignment operator were synthesized for such classes, the left-hand operand would continue to refer to the same object as it did before the assignment. It would not refer to the same object as the right-hand operand. Because this behavior is unlikely to be desired, the synthesized copy-assignment operator is defined as deleted if the class has a reference member.

We'll see in § 13.6.2 (p. 539), § 15.7.2 (p. 624), and § 19.6 (p. 849) that there are other aspects of a class that can cause its copy members to be defined as deleted.

In essence, the copy-control members are synthesized as deleted when it is impossible to copy, assign, or destroy a member of the class.

`private` Copy Control

Prior to the new standard, classes prevented copies by declaring their copy constructor and copy-assignment operator as `private`:

```
class PrivateCopy {
    // no access specifier; following members are private by default; see § 7.2 (p. 268)
    // copy control is private and so is inaccessible to ordinary user code
    PrivateCopy(const PrivateCopy&);
    PrivateCopy &operator=(const PrivateCopy&);
    // other members
public:
    PrivateCopy() = default; // use the synthesized default constructor
    ~PrivateCopy(); // users can define objects of this type but not copy them
};
```

Because the destructor is `public`, users will be able to define `PrivateCopy` objects. However, because the copy constructor and copy-assignment operator are `private`, user code will not be able to copy such objects. However, friends and members of the class can still make copies. To prevent copies by friends and members, we declare these members as `private` but do not define them.

With one exception, which we'll cover in § 15.2.1 (p. 594), it is legal to declare, but not define, a member function (§ 6.1.2, p. 206). An attempt to *use* an undefined

member results in a link-time failure. By declaring (but not defining) a `private` copy constructor, we can forestall any attempt to copy an object of the class type: User code that tries to make a copy will be flagged as an error at compile time; copies made in member functions or friends will result in an error at link time.

Classes that want to prevent copying should define their copy constructor and copy-assignment operators using `= delete` rather than making those members `private`.

Exercises Section 13.1.6

Exercise 13.18: Define an `Employee` class that contains an employee name and a unique employee identifier. Give the class a default constructor and a constructor that takes a `string` representing the employee's name. Each constructor should generate a unique ID by incrementing a `static` data member.

Exercise 13.19: Does your `Employee` class need to define its own versions of the copy-control members? If so, why? If not, why not? Implement whatever copy-control members you think `Employee` needs.

Exercise 13.20: Explain what happens when we copy, assign, or destroy objects of our `TextQuery` and `QueryResult` classes from § 12.3 (p. 484).

Exercise 13.21: Do you think the `TextQuery` and `QueryResult` classes need to define their own versions of the copy-control members? If so, why? If not, why not? Implement whichever copy-control operations you think these classes require.

13.2 Copy Control and Resource Management

Ordinarily, classes that manage resources that do not reside in the class must define the copy-control members. As we saw in § 13.1.4 (p. 504), such classes will need destructors to free the resources allocated by the object. Once a class needs a destructor, it almost surely needs a copy constructor and copy-assignment operator as well.

In order to define these members, we first have to decide what copying an object of our type will mean. In general, we have two choices: We can define the copy operations to make the class behave like a value or like a pointer.

Classes that behave like values have their own state. When we copy a valuelike object, the copy and the original are independent of each other. Changes made to the copy have no effect on the original, and vice versa.

Classes that act like pointers share state. When we copy objects of such classes, the copy and the original use the same underlying data. Changes made to the copy also change the original, and vice versa.

Of the library classes we've used, the library containers and `string` class have valuelike behavior. Not surprisingly, the `shared_ptr` class provides pointerlike behavior, as does our `StrBlob` class (§ 12.1.1, p. 456). The IO types and

unique_ptr do not allow copying or assignment, so they provide neither valuelike nor pointerlike behavior.

To illustrate these two approaches, we'll define the copy-control members for the HasPtr class used in the exercises. First, we'll make the class act like a value; then we'll reimplement the class making it behave like a pointer.

Our HasPtr class has two members, an int and a pointer to string. Ordinarily, classes copy members of built-in type (other than pointers) directly; such members are values and hence ordinarily ought to behave like values. What we do when we copy the pointer member determines whether a class like HasPtr has valuelike or pointerlike behavior.

EXERCISES SECTION 13.2

Exercise 13.22: Assume that we want HasPtr to behave like a value. That is, each object should have its own copy of the string to which the objects point. We'll show the definitions of the copy-control members in the next section. However, you already know everything you need to know to implement these members. Write the HasPtr copy constructor and copy-assignment operator before reading on.

13.2.1 Classes That Act Like Values

To provide valuelike behavior, each object has to have its own copy of the resource that the class manages. That means each HasPtr object must have its own copy of the string to which ps points. To implement valuelike behavior HasPtr needs

- A copy constructor that copies the string, not just the pointer
- A destructor to free the string
- A copy-assignment operator to free the object's existing string and copy the string from its right-hand operand

The valuelike version of HasPtr is

```
class HasPtr {
public:
    HasPtr(const std::string &s = std::string()):
        ps(new std::string(s)), i(0) { }
    //  each HasPtr has its own copy of the string to which ps points
    HasPtr(const HasPtr &p):
        ps(new std::string(*p.ps)), i(p.i) { }
    HasPtr& operator=(const HasPtr &);
    ~HasPtr() { delete ps; }
private:
    std::string *ps;
    int    i;
};
```

Our class is simple enough that we've defined all but the assignment operator in the class body. The first constructor takes an (optional) `string` argument. That constructor dynamically allocates its own copy of that `string` and stores a pointer to that `string` in `ps`. The copy constructor also allocates its own, separate copy of the `string`. The destructor frees the memory allocated in its constructors by executing `delete` on the pointer member, `ps`.

Valuelike Copy-Assignment Operator

Assignment operators typically combine the actions of the destructor and the copy constructor. Like the destructor, assignment destroys the left-hand operand's resources. Like the copy constructor, assignment copies data from the right-hand operand. However, it is crucially important that these actions be done in a sequence that is correct even if an object is assigned to itself. Moreover, when possible, we should also write our assignment operators so that they will leave the left-hand operand in a sensible state should an exception occur (§ 5.6.2, p. 196).

In this case, we can handle self-assignment—and make our code safe should an exception happen—by first copying the right-hand side. After the copy is made, we'll free the left-hand side and update the pointer to point to the newly allocated `string`:

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
    auto newp = new string(*rhs.ps);  // copy the underlying string
    delete ps;        // free the old memory
    ps = newp;        // copy data from rhs into this object
    i = rhs.i;
    return *this;     // return this object
}
```

In this assignment operator, we quite clearly first do the work of the constructor: The initializer of `newp` is identical to the initializer of `ps` in `HasPtr`'s copy constructor. As in the destructor, we next `delete` the `string` to which `ps` currently points. What remains is to copy the pointer to the newly allocated `string` and the `int` value from `rhs` into this object.

KEY CONCEPT: ASSIGNMENT OPERATORS

There are two points to keep in mind when you write an assignment operator:

- **Assignment operators must work correctly if an object is assigned to itself.**
- **Most assignment operators share work with the destructor and copy constructor.**

A good pattern to use when you write an assignment operator is to first copy the right-hand operand into a local temporary. *After* the copy is done, it is safe to destroy the existing members of the left-hand operand. Once the left-hand operand is destroyed, copy the data from the temporary into the members of the left-hand operand.

To illustrate the importance of guarding against self-assignment, consider what would happen if we wrote the assignment operator as

```
// WRONG way to write an assignment operator!
HasPtr&
HasPtr::operator=(const HasPtr &rhs)
{
    delete ps;  // frees the string to which this object points
    // if rhs and *this are the same object, we're copying from deleted memory!
    ps = new string(*(rhs.ps));
    i = rhs.i;
    return *this;
}
```

If `rhs` and this object are the same object, deleting `ps` frees the `string` to which both `*this` and `rhs` point. When we attempt to copy `*(rhs.ps)` in the `new` expression, that pointer points to invalid memory. What happens is undefined.

It is crucially important for assignment operators to work correctly, even when an object is assigned to itself. A good way to do so is to copy the right-hand operand before destroying the left-hand operand.

Exercises Section 13.2.1

Exercise 13.23: Compare the copy-control members that you wrote for the solutions to the previous section's exercises to the code presented here. Be sure you understand the differences, if any, between your code and ours.

Exercise 13.24: What would happen if the version of `HasPtr` in this section didn't define a destructor? What if `HasPtr` didn't define the copy constructor?

Exercise 13.25: Assume we want to define a version of `StrBlob` that acts like a value. Also assume that we want to continue to use a `shared_ptr` so that our `StrBlobPtr` class can still use a `weak_ptr` to the `vector`. Your revised class will need a copy constructor and copy-assignment operator but will not need a destructor. Explain what the copy constructor and copy-assignment operators must do. Explain why the class does not need a destructor.

Exercise 13.26: Write your own version of the `StrBlob` class described in the previous exercise.

13.2.2 Defining Classes That Act Like Pointers

For our `HasPtr` class to act like a pointer, we need the copy constructor and copy-assignment operator to copy the pointer member, not the `string` to which that pointer points. Our class will still need its own destructor to free the memory allocated by the constructor that takes a `string` (§ 13.1.4, p. 504). In this case, though, the destructor cannot unilaterally free its associated `string`. It can do so only when the last `HasPtr` pointing to that `string` goes away.

The easiest way to make a class act like a pointer is to use `shared_ptrs` to manage the resources in the class. Copying (or assigning) a `shared_ptr` copies

(assigns) the pointer to which the `shared_ptr` points. The `shared_ptr` class itself keeps track of how many users are sharing the pointed-to object. When there are no more users, the `shared_ptr` class takes care of freeing the resource.

However, sometimes we want to manage a resource directly. In such cases, it can be useful to use a **reference count** (§ 12.1.1, p. 452). To show how reference counting works, we'll redefine `HasPtr` to provide pointerlike behavior, but we will do our own reference counting.

Reference Counts

Reference counting works as follows:

- In addition to initializing the object, each constructor (other than the copy constructor) creates a counter. This counter will keep track of how many objects share state with the object we are creating. When we create an object, there is only one such object, so we initialize the counter to 1.
- The copy constructor does not allocate a new counter; instead, it copies the data members of its given object, including the counter. The copy constructor increments this shared counter, indicating that there is another user of that object's state.
- The destructor decrements the counter, indicating that there is one less user of the shared state. If the count goes to zero, the destructor deletes that state.
- The copy-assignment operator increments the right-hand operand's counter and decrements the counter of the left-hand operand. If the counter for the left-hand operand goes to zero, there are no more users. In this case, the copy-assignment operator must destroy the state of the left-hand operand.

The only wrinkle is deciding where to put the reference count. The counter cannot be a direct member of a `HasPtr` object. To see why, consider what happens in the following example:

```
HasPtr p1("Hiya!");
HasPtr p2(p1);  // p1 and p2 point to the same string
HasPtr p3(p1);  // p1, p2, and p3 all point to the same string
```

If the reference count is stored in each object, how can we update it correctly when `p3` is created? We could increment the count in `p1` and copy that count into `p3`, but how would we update the counter in `p2`?

One way to solve this problem is to store the counter in dynamic memory. When we create an object, we'll also allocate a new counter. When we copy or assign an object, we'll copy the pointer to the counter. That way the copy and the original will point to the same counter.

Defining a Reference-Counted Class

Using a reference count, we can write the pointerlike version of `HasPtr` as follows:

```
class HasPtr {
public:
    // constructor allocates a new string and a new counter, which it sets to 1
    HasPtr(const std::string &s = std::string()):
      ps(new std::string(s)), i(0), use(new std::size_t(1)) {}
    // copy constructor copies all three data members and increments the counter
    HasPtr(const HasPtr &p):
        ps(p.ps), i(p.i), use(p.use) { ++*use; }
    HasPtr& operator=(const HasPtr&);
    ~HasPtr();
private:
    std::string *ps;
    int    i;
    std::size_t *use;  // member to keep track of how many objects share *ps
};
```

Here, we've added a new data member named `use` that will keep track of how many objects share the same `string`. The constructor that takes a `string` allocates this counter and initializes it to `1`, indicating that there is one user of this object's `string` member.

Pointerlike Copy Members "Fiddle" the Reference Count

When we copy or assign a `HasPtr` object, we want the copy and the original to point to the same `string`. That is, when we copy a `HasPtr`, we'll copy `ps` itself, not the `string` to which `ps` points. When we make a copy, we also increment the counter associated with that `string`.

The copy constructor (which we defined inside the class) copies all three members from its given `HasPtr`. This constructor also increments the `use` member, indicating that there is another user for the `string` to which `ps` and `p.ps` point.

The destructor cannot unconditionally `delete ps`—there might be other objects pointing to that memory. Instead, the destructor decrements the reference count, indicating that one less object shares the `string`. If the counter goes to zero, then the destructor frees the memory to which both `ps` and `use` point:

```
HasPtr::~HasPtr()
{
    if (--*use == 0) {    // if the reference count goes to 0
        delete ps;        // delete the string
        delete use;       // and the counter
    }
}
```

The copy-assignment operator, as usual, does the work common to the copy constructor and to the destructor. That is, the assignment operator must increment the counter of the right-hand operand (i.e., the work of the copy constructor) and decrement the counter of the left-hand operand, deleting the memory used if appropriate (i.e., the work of the destructor).

Also, as usual, the operator must handle self-assignment. We do so by incrementing the count in `rhs` before decrementing the count in the left-hand object.

That way if both objects are the same, the counter will have been incremented before we check to see if `ps` (and `use`) should be deleted:

```
HasPtr& HasPtr::operator=(const HasPtr &rhs)
{
    ++*rhs.use;   // increment the use count of the right-hand operand
    if (--*use == 0) {    // then decrement this object's counter
        delete ps;        // if no other users
        delete use;       // free this object's allocated members
    }
    ps = rhs.ps;          // copy data from rhs into this object
    i = rhs.i;
    use = rhs.use;
    return *this;         // return this object
}
```

Exercises Section 13.2.2

Exercise 13.27: Define your own reference-counted version of `HasPtr`.

Exercise 13.28: Given the following classes, implement a default constructor and the necessary copy-control members.

(a)
```
class TreeNode {
  private:
    std::string value;
    int         count;
    TreeNode    *left;
    TreeNode    *right;
};
```

(b)
```
class BinStrTree {
  private:
    TreeNode *root;
};
```

13.3 Swap

In addition to defining the copy-control members, classes that manage resources often also define a function named `swap` (§ 9.2.5, p. 339). Defining `swap` is particularly important for classes that we plan to use with algorithms that reorder elements (§ 10.2.3, p. 383). Such algorithms call `swap` whenever they need to exchange two elements.

If a class defines its own `swap`, then the algorithm uses that class-specific version. Otherwise, it uses the `swap` function defined by the library. Although, as usual, we don't know how `swap` is implemented, conceptually it's easy to see that swapping two objects involves a copy and two assignments. For example, code to swap two objects of our valuelike `HasPtr` class (§ 13.2.1, p. 511) might look something like:

```
HasPtr temp = v1; // make a temporary copy of the value of v1
v1 = v2;          // assign the value of v2 to v1
v2 = temp;        // assign the saved value of v1 to v2
```

This code copies the `string` that was originally in `v1` twice—once when the `HasPtr` copy constructor copies `v1` into `temp` and again when the assignment operator assigns `temp` to `v2`. It also copies the `string` that was originally in `v2` when it assigns `v2` to `v1`. As we've seen, copying a valuelike `HasPtr` allocates a new `string` and copies the `string` to which the `HasPtr` points.

In principle, none of this memory allocation is necessary. Rather than allocating new copies of the `string`, we'd like `swap` to swap the pointers. That is, we'd like swapping two `HasPtr`s to execute as:

```
string *temp = v1.ps; // make a temporary copy of the pointer in v1.ps
v1.ps = v2.ps;        // assign the pointer in v2.ps to v1.ps
v2.ps = temp;         // assign the saved pointer in v1.ps to v2.ps
```

Writing Our Own `swap` Function

We can override the default behavior of `swap` by defining a version of `swap` that operates on our class. The typical implementation of `swap` is:

```
class HasPtr {
    friend void swap(HasPtr&, HasPtr&);
    // other members as in § 13.2.1 (p. 511)
};
inline
void swap(HasPtr &lhs, HasPtr &rhs)
{
    using std::swap;
    swap(lhs.ps, rhs.ps); // swap the pointers, not the string data
    swap(lhs.i, rhs.i);   // swap the int members
}
```

We start by declaring `swap` as a `friend` to give it access to `HasPtr`'s (`private`) data members. Because `swap` exists to optimize our code, we've defined `swap` as an `inline` function (§ 6.5.2, p. 238). The body of `swap` calls `swap` on each of the data members of the given object. In this case, we first `swap` the pointers and then the `int` members of the objects bound to `rhs` and `lhs`.

Unlike the copy-control members, `swap` is never necessary. However, defining `swap` can be an important optimization for classes that allocate resources.

`swap` Functions Should Call `swap`, Not `std::swap`

There is one important subtlety in this code: Although it doesn't matter in this particular case, it is essential that `swap` functions call `swap` and not `std::swap`. In the `HasPtr` function, the data members have built-in types. There is no type-specific version of `swap` for the built-in types. In this case, these calls will invoke the library `std::swap`.

However, if a class has a member that has its own type-specific `swap` function, calling `std::swap` would be a mistake. For example, assume we had another class named `Foo` that has a member named `h`, which has type `HasPtr`. If we did

not write a `Foo` version of `swap`, then the library version of `swap` would be used. As we've already seen, the library `swap` makes unnecessary copies of the `string`s managed by `HasPtr`.

We can avoid these copies by writing a `swap` function for `Foo`. However, if we wrote the `Foo` version of `swap` as:

```
void swap(Foo &lhs, Foo &rhs)
{
    // WRONG: this function uses the library version of swap, not the HasPtr version
    std::swap(lhs.h, rhs.h);
    // swap other members of type Foo
}
```

this code would compile and execute. However, there would be no performance difference between this code and simply using the default version of `swap`. The problem is that we've explicitly requested the library version of `swap`. However, we don't want the version in `std`; we want the one defined for `HasPtr` objects.

The right way to write this `swap` function is:

```
void swap(Foo &lhs, Foo &rhs)
{
    using std::swap;
    swap(lhs.h, rhs.h);   // uses the HasPtr version of swap
    // swap other members of type Foo
}
```

Each call to `swap` must be unqualified. That is, each call should be to `swap`, not `std::swap`. For reasons we'll explain in § 16.3 (p. 697), if there is a type-specific version of `swap`, that version will be a better match than the one defined in `std`. As a result, if there is a type-specific version of `swap`, calls to `swap` will match that type-specific version. If there is no type-specific version, then—assuming there is a `using` declaration for `swap` in scope—calls to `swap` will use the version in `std`.

Very careful readers may wonder why the `using` declaration inside `swap` does not hide the declarations for the `HasPtr` version of `swap` (§ 6.4.1, p. 234). We'll explain the reasons for why this code works in § 18.2.3 (p. 798).

Using `swap` in Assignment Operators

Classes that define `swap` often use `swap` to define their assignment operator. These operators use a technique known as **copy and swap**. This technique *swaps* the left-hand operand with a *copy* of the right-hand operand:

```
// note rhs is passed by value, which means the HasPtr copy constructor
// copies the string in the right-hand operand into rhs
HasPtr& HasPtr::operator=(HasPtr rhs)
{
    // swap the contents of the left-hand operand with the local variable rhs
    swap(*this, rhs); // rhs now points to the memory this object had used
    return *this;     // rhs is destroyed, which deletes the pointer in rhs
}
```

In this version of the assignment operator, the parameter is not a reference. Instead, we pass the right-hand operand by value. Thus, `rhs` is a copy of the right-hand operand. Copying a `HasPtr` allocates a new copy of that object's `string`.

In the body of the assignment operator, we call `swap`, which swaps the data members of `rhs` with those in `*this`. This call puts the pointer that had been in the left-hand operand into `rhs`, and puts the pointer that was in `rhs` into `*this`. Thus, after the `swap`, the pointer member in `*this` points to the newly allocated `string` that is a copy of the right-hand operand.

When the assignment operator finishes, `rhs` is destroyed and the `HasPtr` destructor is run. That destructor `delete`s the memory to which `rhs` now points, thus freeing the memory to which the left-hand operand had pointed.

The interesting thing about this technique is that it automatically handles self assignment and is automatically exception safe. By copying the right-hand operand before changing the left-hand operand, it handles self assignment in the same was as we did in our original assignment operator (§ 13.2.1, p. 512). It manages exception safety in the same way as the original definition as well. The only code that might throw is the `new` expression inside the copy constructor. If an exception occurs, it will happen before we have changed the left-hand operand.

Assignment operators that use copy and swap are automatically exception safe and correctly handle self-assignment.

Exercises Section 13.3

Exercise 13.29: Explain why the calls to `swap` inside `swap(HasPtr&, HasPtr&)` do not cause a recursion loop.

Exercise 13.30: Write and test a `swap` function for your valuelike version of `HasPtr`. Give your `swap` a print statement that notes when it is executed.

Exercise 13.31: Give your class a `<` operator and define a `vector` of `HasPtr`s. Give that `vector` some elements and then `sort` the `vector`. Note when `swap` is called.

Exercise 13.32: Would the pointerlike version of `HasPtr` benefit from defining a `swap` function? If so, what is the benefit? If not, why not?

13.4 A Copy-Control Example

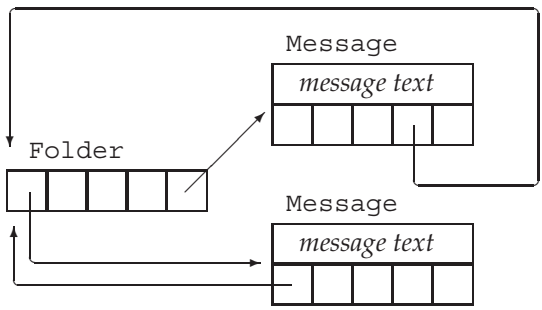
Although copy control is most often needed for classes that allocate resources, resource management is not the only reason why a class might need to define these members. Some classes have bookkeeping or other actions that the copy-control members must perform.

As an example of a class that needs copy control in order to do some bookkeeping, we'll sketch out two classes that might be used in a mail-handling application. These classes, `Message` and `Folder`, represent, respectively, email (or other kinds

of) messages, and directories in which a message might appear. Each `Message` can appear in multiple `Folders`. However, there will be only one copy of the contents of any given `Message`. That way, if the contents of a `Message` are changed, those changes will appear when we view that `Message` from any of its `Folders`.

To keep track of which `Messages` are in which `Folders`, each `Message` will store a `set` of pointers to the `Folders` in which it appears, and each `Folder` will contain a `set` of pointers to its `Messages`. Figure 13.1 illustrates this design.

Figure 13.1: `Message` and `Folder` Class Design



Our `Message` class will provide `save` and `remove` operations to add or remove a `Message` from a specified `Folder`. To create a new `Message`, we will specify the contents of the message but no `Folder`. To put a `Message` in a particular `Folder`, we must call `save`.

When we copy a `Message`, the copy and the original will be distinct `Messages`, but both `Messages` should appear in the same `set` of `Folders`. Thus, copying a `Message` will copy the contents and the `set` of `Folder` pointers. It must also add a pointer to the newly created `Message` to each of those `Folders`.

When we destroy a `Message`, that `Message` no longer exists. Therefore, destroying a `Message` must remove pointers to that `Message` from the `Folders` that had contained that `Message`.

When we assign one `Message` to another, we'll replace the `contents` of the left-hand `Message` with those in the right-hand side. We must also update the `set` of `Folders`, removing the left-hand `Message` from its previous `Folders` and adding that `Message` to the `Folders` in which the right-hand `Message` appears.

Looking at this list of operations, we can see that both the destructor and the copy-assignment operator have to remove this `Message` from the `Folders` that point to it. Similarly, both the copy constructor and the copy-assignment operator add a `Message` to a given list of `Folders`. We'll define a pair of `private` utility functions to do these tasks.

The copy-assignment operator often does the same work as is needed in the copy constructor and destructor. In such cases, the common work should be put in `private` utility functions.

The `Folder` class will need analogous copy control members to add or remove itself from the `Messages` it stores.

We'll leave the design and implementation of the `Folder` class as an exercise. However, we'll assume that the `Folder` class has members named `addMsg` and `remMsg` that do whatever work is need to add or remove this `Message`, respectively, from the set of messages in the given `Folder`.

The `Message` Class

Given this design, we can write our `Message` class as follows:

```
class Message {
    friend class Folder;
public:
    // folders is implicitly initialized to the empty set
    explicit Message(const std::string &str = ""):
        contents(str) { }
    // copy control to manage pointers to this Message
    Message(const Message&);              // copy constructor
    Message& operator=(const Message&);   // copy assignment
    ~Message();                           // destructor
    // add/remove this Message from the specified Folder's set of messages
    void save(Folder&);
    void remove(Folder&);
private:
    std::string contents;       // actual message text
    std::set<Folder*> folders; // Folders that have this Message
    // utility functions used by copy constructor, assignment, and destructor
    // add this Message to the Folders that point to the parameter
    void add_to_Folders(const Message&);
    // remove this Message from every Folder in folders
    void remove_from_Folders();
};
```

The class defines two data members: `contents`, to store the message text, and `folders`, to store pointers to the `Folders` in which this `Message` appears. The constructor that takes a `string` copies the given `string` into `contents` and (implicitly) initializes `folders` to the empty set. Because this constructor has a default argument, it is also the `Message` default constructor (§ 7.5.1, p. 290).

The `save` and `remove` Members

Aside from copy control, the `Message` class has only two `public` members: `save`, which puts the `Message` in the given `Folder`, and `remove`, which takes it out:

```
void Message::save(Folder &f)
{
    folders.insert(&f); // add the given Folder to our list of Folders
    f.addMsg(this);     // add this Message to f's set of Messages
}
```

```
void Message::remove(Folder &f)
{
    folders.erase(&f); //  take the given Folder out of our list of Folders
    f.remMsg(this);    //  remove this Message to f's set of Messages
}
```

To save (or remove) a `Message` requires updating the `folders` member of the `Message`. When we `save` a `Message`, we store a pointer to the given `Folder`; when we `remove` a `Message`, we remove that pointer.

These operations must also update the given `Folder`. Updating a `Folder` is a job that the `Folder` class controls through its `addMsg` and `remMsg` members, which will add or remove a pointer to a given `Message`, respectively.

Copy Control for the `Message` Class

When we copy a `Message`, the copy should appear in the same `Folders` as the original `Message`. As a result, we must traverse the `set` of `Folder` pointers adding a pointer to the new `Message` to each `Folder` that points to the original `Message`. Both the copy constructor and the copy-assignment operator will need to do this work, so we'll define a function to do this common processing:

```
//  add this Message to Folders that point to m
void Message::add_to_Folders(const Message &m)
{
    for (auto f : m.folders) //  for each Folder that holds m
        f->addMsg(this); //  add a pointer to this Message to that Folder
}
```

Here we call `addMsg` on each `Folder` in `m.folders`. The `addMsg` function will add a pointer to this `Message` to that `Folder`.

The `Message` copy constructor copies the data members of the given object:

```
Message::Message(const Message &m):
    contents(m.contents), folders(m.folders)
{
    add_to_Folders(m); //  add this Message to the Folders that point to m
}
```

and calls `add_to_Folders` to add a pointer to the newly created `Message` to each `Folder` that contains the original `Message`.

The `Message` Destructor

When a `Message` is destroyed, we must remove this `Message` from the `Folders` that point to it. This work is shared with the copy-assignment operator, so we'll define a common function to do it:

```
void Message::remove_from_Folders()
{
    for (auto f : folders) //  for each pointer in folders
        f->remMsg(this);   //  remove this Message from that Folder
    folders.clear();       //  no Folder points to this Message
}
```

The implementation of the `remove_from_Folders` function is similar to that of `add_to_Folders`, except that it uses `remMsg` to remove the current `Message`.

Given the `remove_from_Folders` function, writing the destructor is trivial:

```
Message::~Message()
{
    remove_from_Folders();
}
```

The call to `remove_from_Folders` ensures that no `Folder` has a pointer to the `Message` we are destroying. The compiler automatically invokes the `string` destructor to free `contents` and the `set` destructor to clean up the memory used by those members.

`Message` Copy-Assignment Operator

In common with most assignment operators, our `Folder` copy-assignment operator must do the work of the copy constructor and the destructor. As usual, it is crucial that we structure our code to execute correctly even if the left- and right-hand operands happen to be the same object.

In this case, we protect against self-assignment by removing pointers to this `Message` from the `folders` of the left-hand operand before inserting pointers in the `folders` in the right-hand operand:

```
Message& Message::operator=(const Message &rhs)
{
    // handle self-assignment by removing pointers before inserting them
    remove_from_Folders();     // update existing Folders
    contents = rhs.contents;   // copy message contents from rhs
    folders = rhs.folders;     // copy Folder pointers from rhs
    add_to_Folders(rhs);       // add this Message to those Folders
    return *this;
}
```

If the left- and right-hand operands are the same object, then they have the same address. Had we called `remove_from_folders` after calling `add_to_folders`, we would have removed this `Message` from all of its corresponding `Folders`.

A `swap` Function for `Message`

The library defines versions of `swap` for both `string` and `set` (§ 9.2.5, p. 339). As a result, our `Message` class will benefit from defining its own version of `swap`. By defining a `Message`-specific version of `swap`, we can avoid extraneous copies of the `contents` and `folders` members.

However, our `swap` function must also manage the `Folder` pointers that point to the swapped `Messages`. After a call such as `swap(m1, m2)`, the `Folders` that had pointed to `m1` must now point to `m2`, and vice versa.

We'll manage the `Folder` pointers by making two passes through each of the `folders` members. The first pass will remove the `Messages` from their respective `Folders`. We'll next call `swap` to swap the data members. We'll make the second pass through `folders` this time adding pointers to the swapped `Messages`:

```
void swap(Message &lhs, Message &rhs)
{
    using std::swap;   // not strictly needed in this case, but good habit
    // remove pointers to each Message from their (original) respective Folders
    for (auto f: lhs.folders)
        f->remMsg(&lhs);
    for (auto f: rhs.folders)
        f->remMsg(&rhs);
    // swap the contents and Folder pointer sets
    swap(lhs.folders, rhs.folders);    // uses swap(set&, set&)
    swap(lhs.contents, rhs.contents); // swap(string&, string&)
    // add pointers to each Message to their (new) respective Folders
    for (auto f: lhs.folders)
        f->addMsg(&lhs);
    for (auto f: rhs.folders)
        f->addMsg(&rhs);
}
```

Exercises Section 13.4

Exercise 13.33: Why is the parameter to the `save` and `remove` members of `Message` a `Folder&`? Why didn't we define that parameter as `Folder`? Or `const Folder&`?

Exercise 13.34: Write the `Message` class as described in this section.

Exercise 13.35: What would happen if `Message` used the synthesized versions of the copy-control members?

Exercise 13.36: Design and implement the corresponding `Folder` class. That class should hold a `set` that points to the `Messages` in that `Folder`.

Exercise 13.37: Add members to the `Message` class to insert or remove a given `Folder*` into `folders`. These members are analogous to `Folder`'s `addMsg` and `remMsg` operations.

Exercise 13.38: We did not use copy and swap to define the `Message` assignment operator. Why do you suppose this is so?

13.5 Classes That Manage Dynamic Memory

Some classes need to allocate a varying amount of storage at run time. Such classes often can (and if they can, generally should) use a library container to hold their data. For example, our `StrBlob` class uses a `vector` to manage the underlying storage for its elements.

However, this strategy does not work for every class; some classes need to do their own allocation. Such classes generally must define their own copy-control members to manage the memory they allocate.

As an example, we'll implement a simplification of the library `vector` class. Among the simplifications we'll make is that our class will not be a template. Instead, our class will hold `string`s. Thus, we'll call our class `StrVec`.

`StrVec` Class Design

Recall that the `vector` class stores its elements in contiguous storage. To obtain acceptable performance, `vector` preallocates enough storage to hold more elements than are needed (§ 9.4, p. 355). Each `vector` member that adds elements checks whether there is space available for another element. If so, the member constructs an object in the next available spot. If there isn't space left, then the `vector` is reallocated: The `vector` obtains new space, moves the existing elements into that space, frees the old space, and adds the new element.

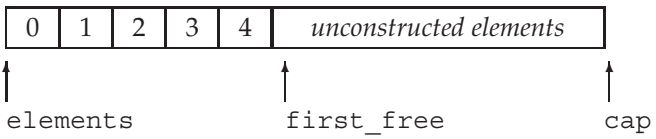
We'll use a similar strategy in our `StrVec` class. We'll use an `allocator` to obtain raw memory (§ 12.2.2, p. 481). Because the memory an `allocator` allocates is unconstructed, we'll use the `allocator`'s `construct` member to create objects in that space when we need to add an element. Similarly, when we remove an element, we'll use the `destroy` member to destroy the element.

Each `StrVec` will have three pointers into the space it uses for its elements:

- `elements`, which points to the first element in the allocated memory
- `first_free`, which points just after the last actual element
- `cap`, which points just past the end of the allocated memory

Figure 13.2 illustrates the meaning of these pointers.

Figure 13.2: `StrVec` Memory Allocation Strategy



In addition to these pointers, `StrVec` will have a `static` data member named `alloc` that is an `allocator<string>`. The `alloc` member will allocate the memory used by a `StrVec`. Our class will also have four utility functions:

- `alloc_n_copy` will allocate space and copy a given range of elements.
- `free` will destroy the constructed elements and deallocate the space.
- `chk_n_alloc` will ensure that there is room to add at least one more element to the `StrVec`. If there isn't room for another element, `chk_n_alloc` will call `reallocate` to get more space.
- `reallocate` will reallocate the `StrVec` when it runs out of space.

Although our focus is on the implementation, we'll also define a few members from `vector`'s interface.

`StrVec` Class Definition

Having sketched the implementation, we can now define our `StrVec` class:

```
// simplified implementation of the memory allocation strategy for a vector-like class
class StrVec {
public:
    StrVec(): // the allocator member is default initialized
      elements(nullptr), first_free(nullptr), cap(nullptr) { }
    StrVec(const StrVec&);                // copy constructor
    StrVec &operator=(const StrVec&); // copy assignment
    ~StrVec();                            // destructor
    void push_back(const std::string&);  // copy the element
    size_t size() const { return first_free - elements; }
    size_t capacity() const { return cap - elements; }
    std::string *begin() const { return elements; }
    std::string *end() const { return first_free; }
    // ...
private:
    static std::allocator<std::string> alloc; // allocates the elements
    void chk_n_alloc() // used by functions that add elements to a StrVec
        { if (size() == capacity()) reallocate(); }
    // utilities used by the copy constructor, assignment operator, and destructor
    std::pair<std::string*, std::string*> alloc_n_copy
        (const std::string*, const std::string*);
    void free();              // destroy the elements and free the space
    void reallocate();        // get more space and copy the existing elements
    std::string *elements;    // pointer to the first element in the array
    std::string *first_free; // pointer to the first free element in the array
    std::string *cap;         // pointer to one past the end of the array
};

// alloc must be defined in the StrVec implmentation file
allocator<string> StrVec::alloc;
```

The class body defines several of its members:

- The default constructor (implicitly) default initializes `alloc` and (explicitly) initializes the pointers to `nullptr`, indicating that there are no elements.
- The `size` member returns the number of elements actually in use, which is equal to `first_free - elements`.
- The `capacity` member returns the number of elements that the `StrVec` can hold, which is equal to `cap - elements`.
- The `chk_n_alloc` causes the `StrVec` to be reallocated when there is no room to add another element, which happens when `cap == first_free`.
- The `begin` and `end` members return pointers to the first (i.e., `elements`) and one past the last constructed element (i.e., `first_free`), respectively.

Using `construct`

The `push_back` function calls `chk_n_alloc` to ensure that there is room for an element. When `chk_n_alloc` returns, `push_back` knows that there is room for the new element. It asks its `alloc` member to `construct` a new last element:

```
void StrVec::push_back(const string& s)
{
    chk_n_alloc(); // ensure that there is room for another element
    // construct a copy of s in the element to which first_free points
    alloc.construct(first_free++, s);
}
```

When we use an `allocator`, we must remember that the memory is *unconstructed* (§ 12.2.2, p. 482). To use this raw memory we must call `construct`, which will construct an object in that memory. The first argument to `construct` must be a pointer to unconstructed space allocated by a call to `allocate`. The remaining arguments determine which constructor to use to construct the object that will go in that space. In this case, there is only one additional argument. That argument has type `string`, so this call uses the `string` copy constructor.

It is worth noting that the call to `construct` also increments `first_free` to indicate that a new element has been constructed. It uses the postfix increment (§ 4.5, p. 147), so this call constructs an object in the current value of `first_free` and increments `first_free` to point to the next, unconstructed element.

The `alloc_n_copy` Member

The `alloc_n_copy` member is called when we copy or assign a `StrVec`. Our `StrVec` class, like `vector`, will have valuelike behavior (§ 13.2.1, p. 511); when we copy or assign a `StrVec`, we have to allocate independent memory and copy the elements from the original to the new `StrVec`.

The `alloc_n_copy` member will allocate enough storage to hold its given range of elements, and will copy those elements into the newly allocated space. This function returns a `pair` (§ 11.2.3, p. 426) of pointers, pointing to the beginning of the new space and just past the last element it copied:

```
pair<string*, string*>
StrVec::alloc_n_copy(const string *b, const string *e)
{
    // allocate space to hold as many elements as are in the range
    auto data = alloc.allocate(e - b);
    // initialize and return a pair constructed from data and
    // the value returned by uninitialized_copy
    return {data, uninitialized_copy(b, e, data)};
}
```

`alloc_n_copy` calculates how much space to allocate by subtracting the pointer to the first element from the pointer one past the last. Having allocated memory, the function next has to construct copies of the given elements in that space.

It does the copy in the return statement, which list initializes the return value (§ 6.3.2, p. 226). The `first` member of the returned `pair` points to the start of the

allocated memory; the `second` is the value returned from `uninitialized_copy` (§ 12.2.2, p. 483). That value will be pointer positioned one element past the last constructed element.

The `free` Member

The `free` member must `destroy` the elements and then deallocate the space this `StrVec` allocated. The `for` loop calls the `allocator` member `destroy` in reverse order, starting with the last constructed element and finishing with the first:

```
void StrVec::free()
{
    //  may not pass deallocate a 0 pointer; if elements is 0, there's no work to do
    if (elements) {
        //  destroy the old elements in reverse order
        for (auto p = first_free; p != elements; /* empty */)
            alloc.destroy(--p);
        alloc.deallocate(elements, cap - elements);
    }
}
```

The `destroy` function runs the `string` destructor. The `string` destructor frees whatever storage was allocated by the `string`s themselves.

Once the elements have been destroyed, we free the space that this `StrVec` allocated by calling `deallocate`. The pointer we pass to `deallocate` must be one that was previously generated by a call to `allocate`. Therefore, we first check that `elements` is not null before calling `deallocate`.

Copy-Control Members

Given our `alloc_n_copy` and `free` members, the copy-control members of our class are straightforward. The copy constructor calls `alloc_n_copy`:

```
StrVec::StrVec(const StrVec &s)
{
    //  call alloc_n_copy to allocate exactly as many elements as in s
    auto newdata = alloc_n_copy(s.begin(), s.end());
    elements = newdata.first;
    first_free = cap = newdata.second;
}
```

and assigns the results from that call to the data members. The return value from `alloc_n_copy` is a `pair` of pointers. The `first` pointer points to the first constructed element and the `second` points just past the last constructed element. Because `alloc_n_copy` allocates space for exactly as many elements as it is given, `cap` also points just past the last constructed element.

The destructor calls `free`:

```
StrVec::~StrVec() { free(); }
```

The copy-assignment operator calls `alloc_n_copy` before freeing its existing elements. By doing so it protects against self-assignment:

```
StrVec &StrVec::operator=(const StrVec &rhs)
{
    // call alloc_n_copy to allocate exactly as many elements as in rhs
    auto data = alloc_n_copy(rhs.begin(), rhs.end());
    free();
    elements = data.first;
    first_free = cap = data.second;
    return *this;
}
```

Like the copy constructor, the copy-assignment operator uses the values returned from `alloc_n_copy` to initialize its pointers.

Moving, Not Copying, Elements during Reallocation

Before we write the `reallocate` member, we should think a bit about what it must do. This function will

- Allocate memory for a new, larger array of `string`s
- Construct the first part of that space to hold the existing elements
- Destroy the elements in the existing memory and deallocate that memory

Looking at this list of steps, we can see that reallocating a `StrVec` entails copying each `string` from the old `StrVec` memory to the new. Although we don't know the details of how `string` is implemented, we do know that `string`s have valuelike behavior. When we copy a `string`, the new `string` and the original `string` are independent from each other. Changes made to the original don't affect the copy, and vice versa.

Because `string`s act like values, we can conclude that each `string` must have its own copy of the characters that make up that `string`. Copying a `string` must allocate memory for those characters, and destroying a `string` must free the memory used by that `string`.

Copying a `string` copies the data because ordinarily after we copy a `string`, there are two users of that `string`. However, when `reallocate` copies the `string`s in a `StrVec`, there will be only one user of these `string`s after the copy. As soon as we copy the elements from the old space to the new, we will immediately destroy the original `string`s.

Copying the data in these `string`s is unnecessary. Our `StrVec`'s performance will be *much* better if we can avoid the overhead of allocating and deallocating the `string`s themselves each time we reallocate.

Move Constructors and `std::move`

We can avoid copying the `string`s by using two facilities introduced by the new library. First, several of the library classes, including `string`, define so-called "move constructors." The details of how the `string` move constructor works—like any other detail about the implementation—are not disclosed. However, we do know that move constructors typically operate by "moving" resources from

the given object to the object being constructed. We also know that the library guarantees that the "moved-from" `string` remains in a valid, destructible state. For `string`, we can imagine that each `string` has a pointer to an array of `char`. Presumably the `string` move constructor copies the pointer rather than allocating space for and copying the characters themselves.

The second facility we'll use is a library function named **`move`**, which is defined in the `utility` header. For now, there are two important points to know about `move`. First, for reasons we'll explain in § 13.6.1 (p. 532), when `reallocate` constructs the `string`s in the new memory it must call `move` to signal that it wants to use the `string` move constructor. If it omits the call to `move` the `string` the copy constructor will be used. Second, for reasons we'll cover in § 18.2.3 (p. 798), we usually do not provide a `using` declaration (§ 3.1, p. 82) for `move`. When we use `move`, we call `std::move`, not `move`.

The `reallocate` Member

Using this information, we can now write our `reallocate` member. We'll start by calling `allocate` to allocate new space. We'll double the capacity of the `StrVec` each time we reallocate. If the `StrVec` is empty, we allocate room for one element:

```
void StrVec::reallocate()
{
    // we'll allocate space for twice as many elements as the current size
    auto newcapacity = size() ? 2 * size() : 1;
    // allocate new memory
    auto newdata = alloc.allocate(newcapacity);
    // move the data from the old memory to the new
    auto dest = newdata;  // points to the next free position in the new array
    auto elem = elements; // points to the next element in the old array
    for (size_t i = 0; i != size(); ++i)
        alloc.construct(dest++, std::move(*elem++));
    free();  // free the old space once we've moved the elements
    // update our data structure to point to the new elements
    elements = newdata;
    first_free = dest;
    cap = elements + newcapacity;
}
```

The `for` loop iterates through the existing elements and `constructs` a corresponding element in the new space. We use `dest` to point to the memory in which to construct the new `string`, and use `elem` to point to an element in the original array. We use postfix increment to move the `dest` (and `elem`) pointers one element at a time through these two arrays.

The second argument in the call to `construct` (i.e., the one that determines which constructor to use (§ 12.2.2, p. 482)) is the value returned by `move`. Calling `move` returns a result that causes `construct` to use the `string` move constructor. Because we're using the move constructor, the memory managed by those `string`s will not be copied. Instead, each `string` we construct will take over ownership of the memory from the `string` to which `elem` points.

After moving the elements, we call `free` to destroy the old elements and free the memory that this `StrVec` was using before the call to `reallocate`. The `string`s themselves no longer manage the memory to which they had pointed; responsibility for their data has been moved to the elements in the new `StrVec` memory. We don't know what value the `string`s in the old `StrVec` memory have, but we are guaranteed that it is safe to run the `string` destructor on these objects.

What remains is to update the pointers to address the newly allocated and initialized array. The `first_free` and `cap` pointers are set to denote one past the last constructed element and one past the end of the allocated space, respectively.

Exercises Section 13.5

Exercise 13.39: Write your own version of `StrVec`, including versions of `reserve`, `capacity` (§ 9.4, p. 356), and `resize` (§ 9.3.5, p. 352).

Exercise 13.40: Add a constructor that takes an `initializer_list<string>` to your `StrVec` class.

Exercise 13.41: Why did we use postfix increment in the call to `construct` inside `push_back`? What would happen if it used the prefix increment?

Exercise 13.42: Test your `StrVec` class by using it in place of the `vector<string>` in your `TextQuery` and `QueryResult` classes (§ 12.3, p. 484).

Exercise 13.43: Rewrite the `free` member to use `for_each` and a lambda (§ 10.3.2, p. 388) in place of the `for` loop to `destroy` the elements. Which implementation do you prefer, and why?

Exercise 13.44: Write a class named `String` that is a simplified version of the library `string` class. Your class should have at least a default constructor and a constructor that takes a pointer to a C-style string. Use an `allocator` to allocate memory that your `String` class uses.

13.6 Moving Objects

One of the major features in the new standard is the ability to move rather than copy an object. As we saw in § 13.1.1 (p. 497), copies are made in many circumstances. In some of these circumstances, an object is immediately destroyed after it is copied. In those cases, moving, rather than copying, the object can provide a significant performance boost.

As we've just seen, our `StrVec` class is a good example of this kind of superfluous copy. During reallocation, there is no need to copy—rather than move—the elements from the old memory to the new. A second reason to move rather than copy occurs in classes such as the IO or `unique_ptr` classes. These classes have a resource (such as a pointer or an IO buffer) that may not be shared. Hence, objects of these types can't be copied but can be moved.

Under earlier versions of the language, there was no direct way to move an object. We had to make a copy even if there was no need for the copy. If the objects are large, or if the objects themselves require memory allocation (e.g., `strings`), making a needless copy can be expensive. Similarly, in previous versions of the library, classes stored in a container had to be copyable. Under the new standard, we can use containers on types that cannot be copied so long as they can be moved.

The library containers, `string`, and `shared_ptr` classes support move as well as copy. The IO and `unique_ptr` classes can be moved but not copied.

13.6.1 Rvalue References

C++ 11

To support move operations, the new standard introduced a new kind of reference, an **rvalue reference**. An rvalue reference is a reference that must be bound to an rvalue. An rvalue reference is obtained by using `&&` rather than `&`. As we'll see, rvalue references have the important property that they may be bound only to an object that is about to be destroyed. As a result, we are free to "move" resources from an rvalue reference to another object.

Recall that lvalue and rvalue are properties of an expression (§ 4.1.1, p. 135). Some expressions yield or require lvalues; others yield or require rvalues. Generally speaking, an lvalue expression refers to an object's identity whereas an rvalue expression refers to an object's value.

Like any reference, an rvalue reference is just another name for an object. As we know, we cannot bind regular references—which we'll refer to as **lvalue references** when we need to distinguish them from rvalue references—to expressions that require a conversion, to literals, or to expressions that return an rvalue (§ 2.3.1, p. 51). Rvalue references have the opposite binding properties: We can bind an rvalue reference to these kinds of expressions, but we cannot directly bind an rvalue reference to an lvalue:

```
int i = 42;
int &r = i;              // ok: r refers to i
int &&rr = i;            // error: cannot bind an rvalue reference to an lvalue
int &r2 = i * 42;        // error: i * 42 is an rvalue
const int &r3 = i * 42; // ok: we can bind a reference to const to an rvalue
int &&rr2 = i * 42;      // ok: bind rr2 to the result of the multiplication
```

Functions that return lvalue references, along with the assignment, subscript, dereference, and prefix increment/decrement operators, are all examples of expressions that return lvalues. We can bind an lvalue reference to the result of any of these expressions.

Functions that return a nonreference type, along with the arithmetic, relational, bitwise, and postfix increment/decrement operators, all yield rvalues. We cannot bind an lvalue reference to these expressions, but we can bind either an lvalue reference to `const` or an rvalue reference to such expressions.

Lvalues Persist; Rvalues Are Ephemeral

Looking at the list of lvalue and rvalue expressions, it should be clear that lvalues and rvalues differ from each other in an important manner: Lvalues have persistent state, whereas rvalues are either literals or temporary objects created in the course of evaluating expressions.

Because rvalue references can only be bound to temporaries, we know that

- The referred-to object is about to be destroyed
- There can be no other users of that object

These facts together mean that code that uses an rvalue reference is free to take over resources from the object to which the reference refers.

Rvalue references refer to objects that are about to be destroyed. Hence, we can "steal" state from an object bound to an rvalue reference.

Variables Are Lvalues

Although we rarely think about it this way, a variable is an expression with one operand and no operator. Like any other expression, a variable expression has the lvalue/rvalue property. Variable expressions are lvalues. It may be surprising, but as a consequence, we cannot bind an rvalue reference to a variable defined as an rvalue reference type:

```
int &&rr1 = 42;    // ok: literals are rvalues
int &&rr2 = rr1;   // error: the expression rr1 is an lvalue!
```

Given our previous observation that rvalues represent ephemeral objects, it should not be surprising that a variable is an lvalue. After all, a variable persists until it goes out of scope.

A variable is an lvalue; we cannot directly bind an rvalue reference to a variable *even if that variable was defined as an rvalue reference type*.

The Library `move` Function

Although we cannot directly bind an rvalue reference to an lvalue, we can explicitly cast an lvalue to its corresponding rvalue reference type. We can also obtain an rvalue reference bound to an lvalue by calling a new library function named **`move`**, which is defined in the `utility` header. The `move` function uses facilities that we'll describe in § 16.2.6 (p. 690) to return an rvalue reference to its given object.

C++ 11

```
int &&rr3 = std::move(rr1);  // ok
```

Calling `move` tells the compiler that we have an lvalue that we want to treat as if it were an rvalue. It is essential to realize that the call to `move` promises that we do not intend to use `rr1` again except to assign to it or to destroy it. After a call to `move`, we cannot make any assumptions about the value of the moved-from object.

We can destroy a moved-from object and can assign a new value to it, but we cannot use the value of a moved-from object.

As we've seen, differently from how we use most names from the library, we do not provide a `using` declaration (§ 3.1, p. 82) for `move` (§ 13.5, p. 530). We call `std::move` not `move`. We'll explain the reasons for this usage in § 18.2.3 (p. 798).

Code that uses `move` should use `std::move`, not `move`. Doing so avoids potential name collisions.

Exercises Section 13.6.1

Exercise 13.45: Distinguish between an rvalue reference and an lvalue reference.

Exercise 13.46: Which kind of reference can be bound to the following initializers?

```
int f();
vector<int> vi(100);
int? r1 = f();
int? r2 = vi[0];
int? r3 = r1;
int? r4 = vi[0] * f();
```

Exercise 13.47: Give the copy constructor and copy-assignment operator in your `String` class from exercise 13.44 in § 13.5 (p. 531) a statement that prints a message each time the function is executed.

Exercise 13.48: Define a `vector<String>` and call `push_back` several times on that `vector`. Run your program and see how often `String`s are copied.

13.6.2 Move Constructor and Move Assignment

Like the `string` class (and other library classes), our own classes can benefit from being able to be moved as well as copied. To enable move operations for our own types, we define a move constructor and a move-assignment operator. These members are similar to the corresponding copy operations, but they "steal" resources from their given object rather than copy them.

C++ 11

Like the copy constructor, the move constructor has an initial parameter that is a reference to the class type. Differently from the copy constructor, the reference parameter in the move constructor is an rvalue reference. As in the copy constructor, any additional parameters must all have default arguments.

In addition to moving resources, the move constructor must ensure that the moved-from object is left in a state such that destroying that object will be harmless. In particular, once its resources are moved, the original object must no longer point to those moved resources—responsibility for those resources has been assumed by the newly created object.

As an example, we'll define the `StrVec` move constructor to move rather than copy the elements from one `StrVec` to another:

```
StrVec::StrVec(StrVec &&s) noexcept  // move won't throw any exceptions
  // member initializers take over the resources in s
  : elements(s.elements), first_free(s.first_free), cap(s.cap)
{
    // leave s in a state in which it is safe to run the destructor
    s.elements = s.first_free = s.cap = nullptr;
}
```

We'll explain the use of `noexcept` (which signals that our constructor does not throw any exceptions) shortly, but let's first look at what this constructor does.

Unlike the copy constructor, the move constructor does not allocate any new memory; it takes over the memory in the given `StrVec`. Having taken over the memory from its argument, the constructor body sets the pointers in the given object to `nullptr`. After an object is moved from, that object continues to exist. Eventually, the moved-from object will be destroyed, meaning that the destructor will be run on that object. The `StrVec` destructor calls `deallocate` on `first_free`. If we neglected to change `s.first_free`, then destroying the moved-from object would delete the memory we just moved.

Move Operations, Library Containers, and Exceptions

Because a move operation executes by "stealing" resources, it ordinarily does not itself allocate any resources. As a result, move operations ordinarily will not throw any exceptions. When we write a move operation that cannot throw, we should inform the library of that fact. As we'll see, unless the library knows that our move constructor won't throw, it will do extra work to cater to the possibliity that moving an object of our class type might throw.

One way inform the library is to specify `noexcept` on our constructor. We'll cover `noexcept`, which was introduced by the new standard, in more detail in § 18.1.4 (p. 779). For now what's important to know is that `noexcept` is a way for us to promise that a function does not throw any exceptions. We specify `noexcept` on a function after its parameter list. In a constructor, `noexcept` appears between the parameter list and the `:` that begins the constructor initializer list:

C++ 11

```
class StrVec {
public:
    StrVec(StrVec&&) noexcept;      // move constructor
    // other members as before
};
StrVec::StrVec(StrVec &&s) noexcept : /* member initializers */
{ /* constructor body */ }
```

We must specify `noexcept` on both the declaration in the class header and on the definition if that definition appears outside the class.

Move constructors and move assignment operators that cannot throw exceptions should be marked as `noexcept`.

Understanding why `noexcept` is needed can help deepen our understanding of how the library interacts with objects of the types we write. We need to indicate that a move operation doesn't throw because of two interrelated facts: First, although move operations usually don't throw exceptions, they are permitted to do so. Second, the library containers provide guarantees as to what they do if an exception happens. As one example, `vector` guarantees that if an exception happens when we call `push_back`, the `vector` itself will be left unchanged.

Now let's think about what happens inside `push_back`. Like the corresponding `StrVec` operation (§ 13.5, p. 527), `push_back` on a `vector` might require that the `vector` be reallocated. When a `vector` is reallocated, it moves the elements from its old space to new memory, just as we did in `reallocate` (§ 13.5, p. 530).

As we've just seen, moving an object generally changes the value of the moved-from object. If reallocation uses a move constructor and that constructor throws an exception after moving some but not all of the elements, there would be a problem. The moved-from elements in the old space would have been changed, and the unconstructed elements in the new space would not yet exist. In this case, `vector` would be unable to meet its requirement that the `vector` is left unchanged.

On the other hand, if `vector` uses the copy constructor and an exception happens, it can easily meet this requirement. In this case, while the elements are being constructed in the new memory, the old elements remain unchanged. If an exception happens, `vector` can free the space it allocated (but could not successfully construct) and return. The original `vector` elements still exist.

To avoid this potential problem, `vector` must use a copy constructor instead of a move constructor during reallocation *unless it knows* that the element type's move constructor cannot throw an exception. If we want objects of our type to be moved rather than copied in circumstances such as `vector` reallocation, we must explicity tell the library that our move constructor is safe to use. We do so by marking the move constructor (and move-assignment operator) `noexcept`.

Move-Assignment Operator

The move-assignment operator does the same work as the destructor and the move constructor. As with the move constructor, if our move-assignment operator won't throw any exceptions, we should make it `noexcept`. Like a copy-assignment operator, a move-assignment operator must guard against self-assignment:

```
StrVec &StrVec::operator=(StrVec &&rhs) noexcept
{
    // direct test for self-assignment
    if (this != &rhs) {
        free();                    // free existing elements
        elements = rhs.elements;   // take over resources from rhs
        first_free = rhs.first_free;
        cap = rhs.cap;
        // leave rhs in a destructible state
        rhs.elements = rhs.first_free = rhs.cap = nullptr;
    }
    return *this;
}
```

In this case we check directly whether the `this` pointer and the address of `rhs` are the same. If they are, the right- and left-hand operands refer to the same object and there is no work to do. Otherwise, we free the memory that the left-hand operand had used, and then take over the memory from the given object. As in the move constructor, we set the pointers in `rhs` to `nullptr`.

It may seem surprising that we bother to check for self-assignment. After all, move assignment requires an rvalue for the right-hand operand. We do the check because that rvalue could be the result of calling `move`. As in any other assignment operator, it is crucial that we not free the left-hand resources before using those (possibly same) resources from the right-hand operand.

A Moved-from Object Must Be Destructible

Moving from an object does not destroy that object: Sometime after the move operation completes, the moved-from object will be destroyed. Therefore, when we write a move operation, we must ensure that the moved-from object is in a state in which the destructor can be run. Our `StrVec` move operations meet this requirement by setting the pointer members of the moved-from object to `nullptr`.

In addition to leaving the moved-from object in a state that is safe to destroy, move operations must guarantee that the object remains valid. In general, a valid object is one that can safely be given a new value or used in other ways that do not depend on its current value. On the other hand, move operations have no requirements as to the value that remains in the moved-from object. As a result, our programs should never depend on the value of a moved-from object.

For example, when we move from a library `string` or container object, we know that the moved-from object remains valid. As a result, we can run operations such as as `empty` or `size` on moved-from objects. However, we don't know what result we'll get. We might expect a moved-from object to be empty, but that is not guaranteed.

Our `StrVec` move operations leave the moved-from object in the same state as a default-initialized object. Therefore, all the operations of `StrVec` will continue to run the same way as they do for any other default-initialized `StrVec`. Other classes, with more complicated internal structure, may behave differently.

After a move operation, the "moved-from" object must remain a valid, destructible object but users may make no assumptions about its value.

The Synthesized Move Operations

As it does for the copy constructor and copy-assignment operator, the compiler will synthesize the move constructor and move-assignment operator. However, the conditions under which it synthesizes a move operation are quite different from those in which it synthesizes a copy operation.

Recall that if we do not declare our own copy constructor or copy-assignment operator the compiler *always* synthesizes these operations (§ 13.1.1, p. 497 and § 13.1.2, p. 500). The copy operations are defined either to memberwise copy or assign the object or they are defined as deleted functions.

Differently from the copy operations, for some classes the compiler does not synthesize the move operations *at all*. In particular, if a class defines its own copy constructor, copy-assignment operator, or destructor, the move constructor and move-assignment operator are not synthesized. As a result, some classes do not have a move constructor or a move-assignment operator. As we'll see on page 540, when a class doesn't have a move operation, the corresponding copy operation is used in place of move through normal function matching.

The compiler will synthesize a move constructor or a move-assignment operator *only* if the class doesn't define any of its own copy-control members and if every nonstatic data member of the class can be moved. The compiler can move members of built-in type. It can also move members of a class type if the member's class has the corresponding move operation:

```
// the compiler will synthesize the move operations for X and hasX
struct X {
    int i;          // built-in types can be moved
    std::string s; // string defines its own move operations
};
struct hasX {
    X mem;  // X has synthesized move operations
};
X x, x2 = std::move(x);        // uses the synthesized move constructor
hasX hx, hx2 = std::move(hx); // uses the synthesized move constructor
```

The compiler synthesizes the move constructor and move assignment only if a class does not define any of its own copy-control members and only if all the data members can be moved constructed and move assigned, respectively.

Unlike the copy operations, a move operation is never implicitly defined as a deleted function. However, if we explicitly ask the compiler to generate a move operation by using = default (§ 7.1.4, p. 264), and the compiler is unable to move all the members, then the move operation will be defined as deleted. With one important exception, the rules for when a synthesized move operation is defined as deleted are analogous to those for the copy operations (§ 13.1.6, p. 508):

- Unlike the copy constructor, the move constructor is defined as deleted if the class has a member that defines its own copy constructor but does not also define a move constructor, or if the class has a member that doesn't define its own copy operations and for which the compiler is unable to synthesize a move constructor. Similarly for move-assignment.
- The move constructor or move-assignment operator is defined as deleted if the class has a member whose own move constructor or move-assignment operator is deleted or inaccessible.
- Like the copy constructor, the move constructor is defined as deleted if the destructor is deleted or inaccessible.
- Like the copy-assignment operator, the move-assignment operator is defined as deleted if the class has a const or reference member.

For example, assuming `Y` is a class that defines its own copy constructor but does not also define its own move constructor:

```
// assume Y is a class that defines its own copy constructor but not a move constructor
struct hasY {
    hasY() = default;
    hasY(hasY&&) = default;
    Y mem; // hasY will have a deleted move constructor
};
hasY hy, hy2 = std::move(hy); // error: move constructor is deleted
```

The compiler can copy objects of type `Y` but cannot move them. Class `hasY` explicitly requested a move constructor, which the compiler is unable to generate. Hence, `hasY` will get a deleted move constructor. Had `hasY` omitted the declaration of its move constructor, then the compiler would not synthesize the `hasY` move constructor at all. The move operations are not synthesized if they would otherwise be defined as deleted.

There is one final interaction between move operations and the synthesized copy-control members: Whether a class defines its own move operations has an impact on how the copy operations are synthesized. If the class defines either a move constructor and/or a move-assignment operator, then the synthesized copy constructor and copy-assignment operator for that class will be defined as deleted.

Classes that define a move constructor or move-assignment operator must also define their own copy operations. Otherwise, those members are deleted by default.

Rvalues Are Moved, Lvalues Are Copied . . .

When a class has both a move constructor and a copy constructor, the compiler uses ordinary function matching to determine which constructor to use (§ 6.4, p. 233). Similarly for assignment. For example, in our `StrVec` class the copy versions take a reference to `const StrVec`. As a result, they can be used on any type that can be converted to `StrVec`. The move versions take a `StrVec&&` and can be used only when the argument is a (non`const`) rvalue:

```
StrVec v1, v2;
v1 = v2;                        // v2 is an lvalue; copy assignment
StrVec getVec(istream &);       // getVec returns an rvalue
v2 = getVec(cin);               // getVec(cin) is an rvalue; move assignment
```

In the first assignment, we pass `v2` to the assignment operator. The type of `v2` is `StrVec` and the expression, `v2`, is an lvalue. The move version of assignment is not viable (§ 6.6, p. 243), because we cannot implicitly bind an rvalue reference to an lvalue. Hence, this assignment uses the copy-assignment operator.

In the second assignment, we assign from the result of a call to `getVec`. That expression is an rvalue. In this case, both assignment operators are viable—we can bind the result of `getVec` to either operator's parameter. Calling the copy-assignment operator requires a conversion to `const`, whereas `StrVec&&` is an exact match. Hence, the second assignment uses the move-assignment operator.

. . . But Rvalues Are Copied If There Is No Move Constructor

What if a class has a copy constructor but does not define a move constructor? In this case, the compiler will not synthesize the move constructor, which means the class has a copy constructor but no move constructor. If a class has no move constructor, function matching ensures that objects of that type are copied, even if we attempt to move them by calling `move`:

```
class Foo {
public:
    Foo() = default;
    Foo(const Foo&);   //  copy constructor
    //  other members, but Foo does not define a move constructor
};
Foo x;
Foo y(x);              //  copy constructor; x is an lvalue
Foo z(std::move(x)); //  copy constructor, because there is no move constructor
```

The call to `move(x)` in the initialization of `z` returns a `Foo&&` bound to `x`. The copy constructor for `Foo` is viable because we can convert a `Foo&&` to a `const Foo&`. Thus, the initialization of `z` uses the copy constructor for `Foo`.

It is worth noting that using the copy constructor in place of a move constructor is almost surely safe (and similarly for the assignment operators). Ordinarily, the copy constructor will meet the requirements of the corresponding move constructor: It will copy the given object and leave that original object in a valid state. Indeed, the copy constructor won't even change the value of the original object.

If a class has a usable copy constructor and no move constructor, objects will be "moved" by the copy constructor. Similarly for the copy-assignment operator and move-assignment.

Copy-and-Swap Assignment Operators and Move

The version of our `HasPtr` class that defined a copy-and-swap assignment operator (§ 13.3, p. 518) is a good illustration of the interaction between function matching and move operations. If we add a move constructor to this class, it will effectively get a move assignment operator as well:

```
class HasPtr {
public:
    //  added move constructor
    HasPtr(HasPtr &&p) noexcept : ps(p.ps), i(p.i) {p.ps = 0;}
    //  assignment operator is both the move- and copy-assignment operator
    HasPtr& operator=(HasPtr rhs)
                    { swap(*this, rhs); return *this; }
    //  other members as in § 13.2.1 (p. 511)
};
```

In this version of the class, we've added a move constructor that takes over the values from its given argument. The constructor body sets the pointer member of

the given HasPtr to zero to ensure that it is safe to destroy the moved-from object. Nothing this function does can throw an exception so we mark it as noexcept (§ 13.6.2, p. 535).

Now let's look at the assignment operator. That operator has a nonreference parameter, which means the parameter is copy initialized (§ 13.1.1, p. 497). Depending on the type of the argument, copy initialization uses either the copy constructor or the move constructor; lvalues are copied and rvalues are moved. As a result, this single assignment operator acts as both the copy-assignment and move-assignment operator.

For example, assuming both hp and hp2 are HasPtr objects:

```
hp = hp2; // hp2 is an lvalue; copy constructor used to copy hp2
hp = std::move(hp2); // move constructor moves hp2
```

In the first assignment, the right-hand operand is an lvalue, so the move constructor is not viable. The copy constructor will be used to initialize rhs. The copy constructor will allocate a new string and copy the string to which hp2 points.

In the second assignment, we invoke std::move to bind an rvalue reference to hp2. In this case, both the copy constructor and the move constructor are viable. However, because the argument is an rvalue reference, it is an exact match for the move constructor. The move constructor copies the pointer from hp2. It does not allocate any memory.

Regardless of whether the copy or move constructor was used, the body of the assignment operator swaps the state of the two operands. Swapping a HasPtr exchanges the pointer (and int) members of the two objects. After the swap, rhs will hold a pointer to the string that had been owned by the left-hand side. That string will be destroyed when rhs goes out of scope.

ADVICE: UPDATING THE RULE OF THREE

All five copy-control members should be thought of as a unit: Ordinarily, if a class defines any of these operations, it usually should define them all. As we've seen, some classes *must* define the copy constructor, copy-assignment operator, and destructor to work correctly (§ 13.1.4, p. 504). Such classes typically have a resource that the copy members must copy. Ordinarily, copying a resource entails some amount of overhead. Classes that define the move constructor and move-assignment operator can avoid this overhead in those circumstances where a copy isn't necessary.

Move Operations for the Message Class

Classes that define their own copy constructor and copy-assignment operator generally also benefit by defining the move operations. For example, our Message and Folder classes (§ 13.4, p. 519) should define move operations. By defining move operations, the Message class can use the string and set move operations to avoid the overhead of copying the contents and folders members.

However, in addition to moving the folders member, we must also update each Folder that points to the original Message. We must remove pointers to the old Message and add a pointer to the new one.

Both the move constructor and move-assignment operator need to update the `Folder` pointers, so we'll start by defining an operation to do this common work:

```
// move the Folder pointers from m to this Message
void Message::move_Folders(Message *m)
{
    folders = std::move(m->folders); // uses set move assignment
    for (auto f : folders) {  // for each Folder
        f->remMsg(m);     // remove the old Message from the Folder
        f->addMsg(this); // add this Message to that Folder
    }
    m->folders.clear();  // ensure that destroying m is harmless
}
```

This function begins by moving the `folders` set. By calling `move`, we use the `set` move assignment rather than its copy assignment. Had we omitted the call to `move`, the code would still work, but the copy is unnecessary. The function then iterates through those `Folders`, removing the pointer to the original `Message` and adding a pointer to the new `Message`.

It is worth noting that inserting an element to a `set` might throw an exception—adding an element to a container requires memory to be allocated, which means that a `bad_alloc` exception might be thrown (§ 12.1.2, p. 460). As a result, unlike our `HasPtr` and `StrVec` move operations, the `Message` move constructor and move-assignment operators might throw exceptions. We will not mark them as `noexcept` (§ 13.6.2, p. 535).

The function ends by calling `clear` on `m.folders`. After the `move`, we know that `m.folders` is valid but have no idea what its contents are. Because the `Message` destructor iterates through `folders`, we want to be certain that the `set` is empty.

The `Message` move constructor calls `move` to move the `contents` and default initializes its `folders` member:

```
Message::Message(Message &&m): contents(std::move(m.contents))
{
    move_Folders(&m); // moves folders and updates the Folder pointers
}
```

In the body of the constructor, we call `move_Folders` to remove the pointers to `m` and insert pointers to this `Message`.

The move-assignment operator does a direct check for self-assignment:

```
Message& Message::operator=(Message &&rhs)
{
    if (this != &rhs) {        // direct check for self-assignment
        remove_from_Folders();
        contents = std::move(rhs.contents); // move assignment
        move_Folders(&rhs); // reset the Folders to point to this Message
    }
    return *this;
}
```

As with any assignment operator, the move-assignment operator must destroy the old state of the left-hand operand. In this case, destroying the left-hand operand requires that we remove pointers to this `Message` from the existing `folders`, which we do in the call to `remove_from_Folders`. Having removed itself from its `Folders`, we call `move` to move the `contents` from `rhs` to `this` object. What remains is to call `move_Messages` to update the `Folder` pointers.

Move Iterators

The `reallocate` member of `StrVec` (§ 13.5, p. 530) used a `for` loop to call `construct` to copy the elements from the old memory to the new. As an alternative to writing that loop, it would be easier if we could call `uninitialized_copy` to construct the newly allocated space. However, `uninitialized_copy` does what it says: It copies the elements. There is no analogous library function to "move" objects into unconstructed memory.

Instead, the new library defines a **move iterator** adaptor (§ 10.4, p. 401). A move iterator adapts its given iterator by changing the behavior of the iterator's dereference operator. Ordinarily, an iterator dereference operator returns an lvalue reference to the element. Unlike other iterators, the dereference operator of a move iterator yields an rvalue reference.

C++ 11

We transform an ordinary iterator to a move iterator by calling the library `make_move_iterator` function. This function takes an iterator and returns a move iterator.

All of the original iterator's other operations work as usual. Because these iterators support normal iterator operations, we can pass a pair of move iterators to an algorithm. In particular, we can pass move iterators to `uninitialized_copy`:

```
void StrVec::reallocate()
{
    // allocate space for twice as many elements as the current size
    auto newcapacity = size() ? 2 * size() : 1;
    auto first = alloc.allocate(newcapacity);
    // move the elements
    auto last = uninitialized_copy(make_move_iterator(begin()),
                                   make_move_iterator(end()),
                                   first);
    free();             // free the old space
    elements = first;   // update the pointers
    first_free = last;
    cap = elements + newcapacity;
}
```

`uninitialized_copy` calls `construct` on each element in the input sequence to "copy" that element into the destination. That algorithm uses the iterator dereference operator to fetch elements from the input sequence. Because we passed move iterators, the dereference operator yields an rvalue reference, which means `construct` will use the move constructor to construct the elements.

It is worth noting that standard library makes no guarantees about which algorithms can be used with move iterators and which cannot. Because moving an

object can obliterate the source, you should pass move iterators to algorithms only when you are *confident* that the algorithm does not access an element after it has assigned to that element or passed that element to a user-defined function.

Advice: Don't Be Too Quick to Move

Because a moved-from object has indeterminate state, calling `std::move` on an object is a dangerous operation. When we call `move`, we must be absolutely certain that there can be no other users of the moved-from object.

Judiciously used inside class code, `move` can offer significant performance benefits. Casually used in ordinary user code (as opposed to class implementation code), moving an object is more likely to lead to mysterious and hard-to-find bugs than to any improvement in the performance of the application.

Outside of class implementation code such as move constructors or move-assignment operators, use `std::move` only when you *are certain* that you need to do a move and that the move is guaranteed to be safe.

Exercises Section 13.6.2

Exercise 13.49: Add a move constructor and move-assignment operator to your `StrVec`, `String`, and `Message` classes.

Exercise 13.50: Put print statements in the move operations in your `String` class and rerun the program from exercise 13.48 in § 13.6.1 (p. 534) that used a `vector<String>` to see when the copies are avoided.

Exercise 13.51: Although `unique_ptrs` cannot be copied, in § 12.1.5 (p. 471) we wrote a `clone` function that returned a `unique_ptr` by value. Explain why that function is legal and how it works.

Exercise 13.52: Explain in detail what happens in the assignments of the `HasPtr` objects on page 541. In particular, describe step by step what happens to values of `hp`, `hp2`, and of the `rhs` parameter in the `HasPtr` assignment operator.

Exercise 13.53: As a matter of low-level efficiency, the `HasPtr` assignment operator is not ideal. Explain why. Implement a copy-assignment and move-assignment operator for `HasPtr` and compare the operations executed in your new move-assignment operator versus the copy-and-swap version.

Exercise 13.54: What would happen if we defined a `HasPtr` move-assignment operator but did not change the copy-and-swap operator? Write code to test your answer.

13.6.3 Rvalue References and Member Functions

Member functions other than constructors and assignment can benefit from providing both copy and move versions. Such move-enabled members typically use

the same parameter pattern as the copy/move constructor and the assignment operators—one version takes an lvalue reference to `const`, and the second takes an rvalue reference to non`const`.

For example, the library containers that define `push_back` provide two versions: one that has an rvalue reference parameter and the other a `const` lvalue reference. Assuming `X` is the element type, these containers define:

```
void push_back(const X&); // copy: binds to any kind of X
void push_back(X&&);      // move: binds only to modifiable rvalues of type X
```

We can pass any object that can be converted to type `X` to the first version of `push_back`. This version copies data from its parameter. We can pass only an rvalue that is not `const` to the second version. This version is an exact match (and a better match) for non`const` rvalues and will be run when we pass a modifiable rvalue (§ 13.6.2, p. 539). This version is free to steal resources from its parameter.

Ordinarily, there is no need to define versions of the operation that take a `const X&&` or a (plain) `X&`. Usually, we pass an rvalue reference when we want to "steal" from the argument. In order to do so, the argument must not be `const`. Similarly, copying from an object should not change the object being copied. As a result, there is usually no need to define a version that take a (plain) `X&` parameter.

Overloaded functions that distinguish between moving and copying a parameter typically have one version that takes a `const T&` and one that takes a `T&&`.

As a more concrete example, we'll give our `StrVec` class a second version of `push_back`:

```
class StrVec {
public:
    void push_back(const std::string&);   // copy the element
    void push_back(std::string&&);        // move the element
    // other members as before
};
// unchanged from the original version in § 13.5 (p. 527)
void StrVec::push_back(const string& s)
{
    chk_n_alloc(); // ensure that there is room for another element
    // construct a copy of s in the element to which first_free points
    alloc.construct(first_free++, s);
}
void StrVec::push_back(string &&s)
{
    chk_n_alloc(); // reallocates the StrVec if necessary
    alloc.construct(first_free++, std::move(s));
}
```

These members are nearly identical. The difference is that the rvalue reference version of `push_back` calls `move` to pass its parameter to `construct`. As we've seen, the `construct` function uses the type of its second and subsequent arguments to determine which constructor to use. Because `move` returns an rvalue reference, the

type of the argument to `construct` is `string&&`. Therefore, the `string` move constructor will be used to construct a new last element.

When we call `push_back` the type of the argument determines whether the new element is copied or moved into the container:

```
StrVec vec;  // empty StrVec
string s = "some string or another";
vec.push_back(s);       // calls push_back(const string&)
vec.push_back("done"); // calls push_back(string&&)
```

These calls differ as to whether the argument is an lvalue (`s`) or an rvalue (the temporary `string` created from `"done"`). The calls are resolved accordingly.

Rvalue and Lvalue Reference Member Functions

Ordinarily, we can call a member function on an object, regardless of whether that object is an lvalue or an rvalue. For example:

```
string s1 = "a value", s2 = "another";
auto n = (s1 + s2).find('a');
```

Here, we called the `find` member (§ 9.5.3, p. 364) on the `string` rvalue that results from adding two `string`s. Sometimes such usage can be surprising:

```
s1 + s2 = "wow!";
```

Here we assign to the rvalue result of concatentating these `string`s.

Prior to the new standard, there was no way to prevent such usage. In order to maintain backward compatability, the library classes continue to allow assignment to rvalues, However, we might want to prevent such usage in our own classes. In this case, we'd like to force the left-hand operand (i.e., the object to which `this` points) to be an lvalue.

C++ 11

We indicate the lvalue/rvalue property of `this` in the same way that we define `const` member functions (§ 7.1.2, p. 258); we place a **reference qualifier** after the parameter list:

```
class Foo {
public:
    Foo &operator=(const Foo&) &; // may assign only to modifiable lvalues
    // other members of Foo
};
Foo &Foo::operator=(const Foo &rhs) &
{
    // do whatever is needed to assign rhs to this object
    return *this;
}
```

The reference qualifier can be either `&` or `&&`, indicating that `this` may point to an rvalue or lvalue, respectively. Like the `const` qualifier, a reference qualifier may appear only on a (nonstatic) member function and must appear in both the declaration and definition of the function.

We may run a function qualified by `&` only on an lvalue and may run a function qualified by `&&` only on an rvalue:

```
Foo &retFoo(); // returns a reference; a call to retFoo is an lvalue
Foo retVal();  // returns by value; a call to retVal is an rvalue
Foo i, j;      // i and j are lvalues
i = j;         // ok: i is an lvalue
retFoo() = j;  // ok: retFoo() returns an lvalue
retVal() = j;  // error: retVal() returns an rvalue
i = retVal();  // ok: we can pass an rvalue as the right-hand operand to assignment
```

A function can be both `const` and reference qualified. In such cases, the reference qualifier must follow the `const` qualifier:

```
class Foo {
public:
    Foo someMem() & const;     // error: const qualifier must come first
    Foo anotherMem() const &; // ok: const qualifier comes first
};
```

Overloading and Reference Functions

Just as we can overload a member function based on whether it is `const` (§ 7.3.2, p. 276), we can also overload a function based on its reference qualifier. Moreover, we may overload a function by its reference qualifier and by whether it is a `const` member. As an example, we'll give `Foo` a `vector` member and a function named `sorted` that returns a copy of the `Foo` object in which the `vector` is sorted:

```
class Foo {
public:
    Foo sorted() &&;          // may run on modifiable rvalues
    Foo sorted() const &;     // may run on any kind of Foo
    // other members of Foo
private:
    vector<int> data;
};
// this object is an rvalue, so we can sort in place
Foo Foo::sorted() &&
{
    sort(data.begin(), data.end());
    return *this;
}
// this object is either const or it is an lvalue; either way we can't sort in place
Foo Foo::sorted() const & {
    Foo ret(*this);                              // make a copy
    sort(ret.data.begin(), ret.data.end()); // sort the copy
    return ret;                                  // return the copy
}
```

When we run `sorted` on an rvalue, it is safe to sort the `data` member directly. The object is an rvalue, which means it has no other users, so we can change the object itself. When we run `sorted` on a `const` rvalue or on an lvalue, we can't change this object, so we copy `data` before sorting it.

Overload resolution uses the lvalue/rvalue property of the object that calls `sorted` to determine which version is used:

```
retVal().sorted(); //  retVal() is an rvalue, calls Foo::sorted() &&
retFoo().sorted(); //  retFoo() is an lvalue, calls Foo::sorted() const &
```

When we define `const` memeber functions, we can define two versions that differ only in that one is `const` qualified and the other is not. There is no similar default for reference qualified functions. When we define two or more members that have the same name and the same parameter list, we must provide a reference qualifier on all or none of those functions:

```
class Foo {
public:
    Foo sorted() &&;
    Foo sorted() const; // error: must have reference qualifier
    //  Comp is type alias for the function type (see § 6.7 (p. 249))
    //  that can be used to compare int values
    using Comp = bool(const int&, const int&);
    Foo sorted(Comp*);          //  ok: different parameter list
    Foo sorted(Comp*) const;    //  ok: neither version is reference qualified
};
```

Here the declaration of the `const` version of `sorted` that has no parameters is an error. There is a second version of `sorted` that has no parameters and that function has a reference qualifier, so the `const` version of that function must have a reference qualifier as well. On the other hand, the versions of `sorted` that take a pointer to a comparison operation are fine, because neither function has a qualifier.

If a member function has a reference qualifier, all the versions of that member with the same parameter list must have reference qualifiers.

Exercises Section 13.6.3

Exercise 13.55: Add an rvalue reference version of `push_back` to your `StrBlob`.

Exercise 13.56: What would happen if we defined `sorted` as:

```
Foo Foo::sorted() const & {
    Foo ret(*this);
    return ret.sorted();
}
```

Exercise 13.57: What if we defined `sorted` as:

```
Foo Foo::sorted() const & { return Foo(*this).sorted(); }
```

Exercise 13.58: Write versions of class `Foo` with print statements in their `sorted` functions to test your answers to the previous two exercises.

Chapter Summary

Each class controls what happens when we copy, move, assign, or destroy objects of its type. Special member functions—the copy constructor, move constructor, copy-assignment operator, move-assignment operator, and destructor—define these operations. The move constructor and move-assignment operator take a (usually nonconst) rvalue reference; the copy versions take a (usually const) ordinary lvalue reference.

If a class declares none of these operations, the compiler will define them automatically. If not defined as deleted, these operations memberwise initialize, move, assign, or destroy the object: Taking each nonstatic data member in turn, the synthesized operation does whatever is appropriate to the member's type to move, copy, assign, or destroy that member.

Classes that allocate memory or other resources almost always require that the class define the copy-control members to manage the allocated resource. If a class needs a destructor, then it almost surely needs to define the move and copy constructors and the move- and copy-assignment operators as well.

Defined Terms

copy and swap Technique for writing assignment operators by copying the right-hand operand followed by a call to swap to exchange the copy with the left-hand operand.

copy-assignment operator Version of the assignment operator that takes an object of the same type as its type. Ordinarily, the copy-assignment operator has a parameter that is a reference to const and returns a reference to its object. The compiler synthesizes the copy-assignment operator if the class does not explicitly provide one.

copy constructor Constructor that initializes a new object as a copy of another object of the same type. The copy constructor is applied implicitly to pass objects to or from a function by value. If we do not provide the copy constructor, the compiler synthesizes one for us.

copy control Special members that control what happens when objects of class type are copied, moved, assigned, and destroyed. The compiler synthesizes appropriate definitions for these operations if the class does not otherwise declare them.

copy initialization Form of initialization used when we use = to supply an initializer for a newly created object. Also used when we pass or return an object by value and when we initialize an array or an aggregate class. Copy initialization uses the copy constructor or the move constructor, depending on whether the initializer is an lvalue or an rvalue.

deleted function Function that may not be used. We delete a function by specifying = delete on its declaration. A common use of deleted functions is to tell the compiler not to synthesize the copy and/or move operations for a class.

destructor Special member function that cleans up an object when the object goes out of scope or is deleted. The compiler automatically destroys each data member. Members of class type are destroyed by invoking their destructor; no work is done when destroying members of built-in or compound type. In particular, the object pointed to by a pointer member is not deleted by the destructor.

lvalue reference Reference that can bind to an lvalue.

memberwise copy/assign How the synthesized copy and move constructors and the copy- and move-assignment operators work. Taking each nonstatic data member in turn, the synthesized copy or move constructor initializes each member by copying or moving the corresponding member from the given object; the copy- or move-assignment operators copy-assign or move-assign each member from the right-hand object to the left. Members of built-in or compound type are initialized or assigned directly. Members of class type are initialized or assigned by using the member's corresponding copy/move constructor or copy-/move-assignment operator.

move Library function used to bind an rvalue reference to an lvalue. Calling move implicitly promises that we will not use the moved-from object except to destroy it or assign a new value to it.

move-assignment operator Version of the assignment operator that takes an rvalue reference to its type. Typically, a move-assignment operator moves data from the right-hand operand to the left. After the assignment, it must be safe to run the destructor on the right-hand operand.

move constructor Constructor that takes an rvalue reference to its type. Typically, a move constructor moves data from its parameter into the newly created object. After the move, it must be safe to run the destructor on the given argument.

move iterator Iterator adaptor that generates an iterator that, when dereferenced, yields an rvalue reference.

overloaded operator Function that redefines the meaning of an operator when applied to operand(s) of class type. This chapter showed how to define the assignment operator; Chapter 14 covers overloaded operators in more detail.

reference count Programming technique often used in copy-control members. A reference count keeps track of how many objects share state. Constructors (other than copy/move constructors) set the reference count to 1. Each time a new copy is made the count is incremented. When an object is destroyed, the count is decremented. The assignment operator and the destructor check whether the decremented reference count has gone to zero and, if so, they destroy the object.

reference qualifier Symbol used to indicate that a nonstatic member function can be called on an lvalue or an rvalue. The qualifier, & or &&, follows the parameter list or the const qualifier if there is one. A function qualified by & may be called only on lvalues; a function qualified by && may be called only on rvalues.

rvalue reference Reference to an object that is about to be destroyed.

synthesized assignment operator A version of the copy- or move-assignment operator created (synthesized) by the compiler for classes that do not explicitly define assignment operators. Unless it is defined as deleted, a synthesized assignment operator memberwise assigns (moves) the right-hand operand to the left.

synthesized copy/move constructor A version of the copy or move constructor that is generated by the compiler for classes that do not explicitly define the corresponding constructor. Unless it is defined as deleted, a synthesized copy or move constructor memberwise initializes the new object by copying or moving members from the given object, respectively.

synthesized destructor Version of the destructor created (synthesized) by the compiler for classes that do not explicitly define one. The synthesized destructor has an empty function body.